

DASC7606

Deep Learning

Deep Generative Models

Dr. Bethany Chan
Professor Francis Chin
2025

Deep Generative Models

- **DeepDream**
- Style transfer
- Autoencoder
- Variation Autoencoder (VAE)
- Generative Adversarial Network (GAN)
 - Conditional-GAN
 - Pix2Pix
 - CycleGAN
- Stable Diffusion

Deep Dream Generator

A computer vision program created by Google that uses a convolutional neural network to find and enhance patterns in images, thus generating **machine** hallucinated images. (2020)

Text-prompt-based image generation (2022)

A powerful AI art and **video** generation platform that lets you create stunning visuals using advanced neural networks. (2023)

AI video generation models:

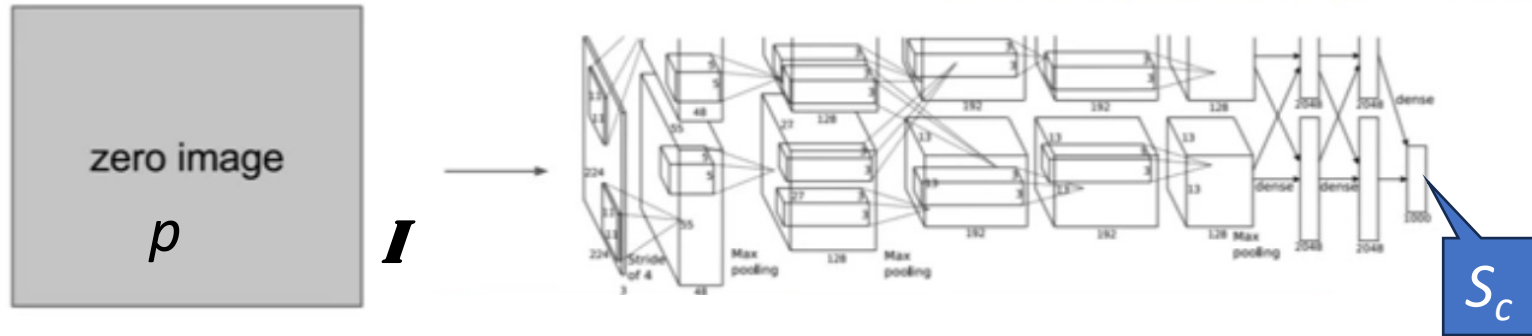
Google's Veo 3, Hailuo AI, MiniMax, Seedance 1.0 Pro, Kling 2.1, and ProVideo.

<https://deepdreamgenerator.com/>

Recall Visualization: Gradient Ascent

Fix all weights of a CNN. Generate a synthetic image that maximizes the activation value of a neuron.

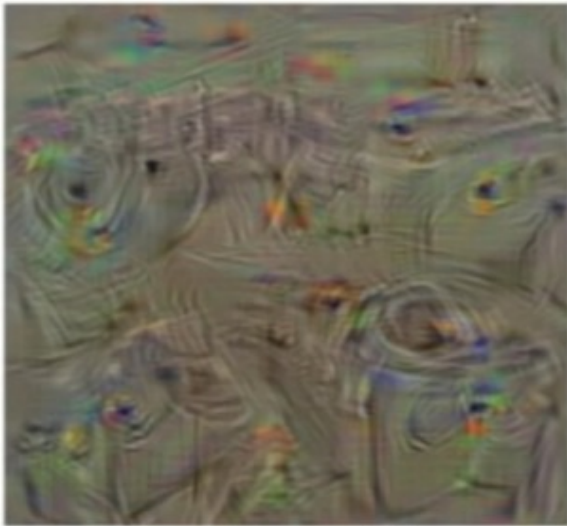
Initialize image to zeros



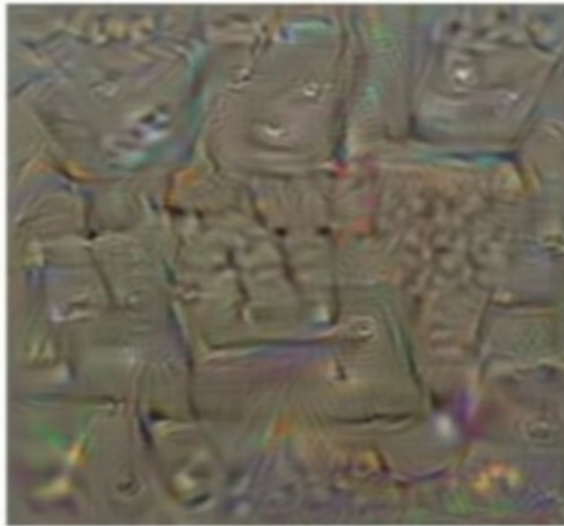
Repeat:

1. Forward image to compute current scores
2. Backprop gradient of neuron value w.r.t. image pixels
3. Make a small update to the image pixel p

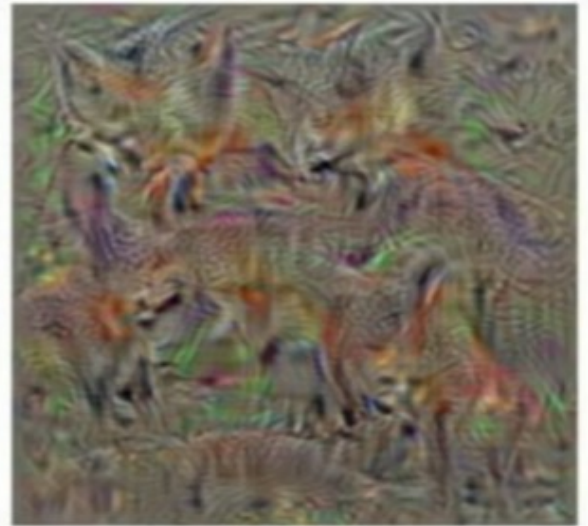
Generated Images with Gradient Ascent



washing machine



computer keyboard



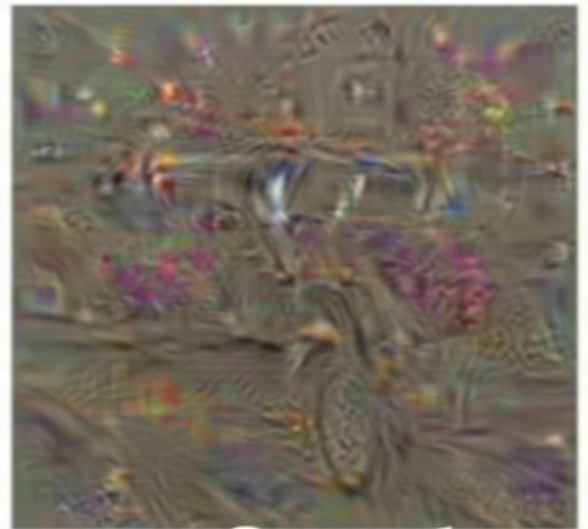
kit fox



goose



ostrich

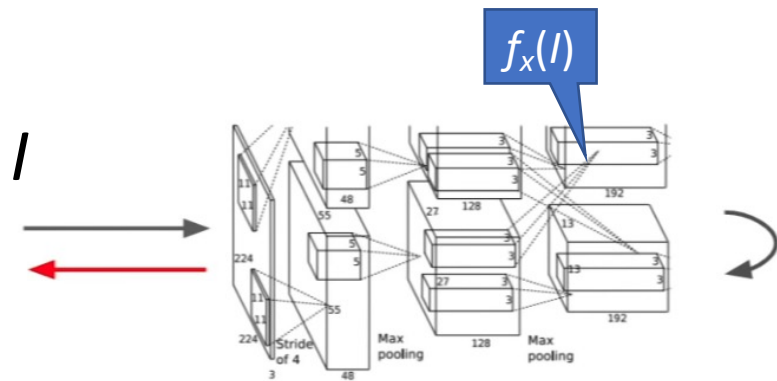
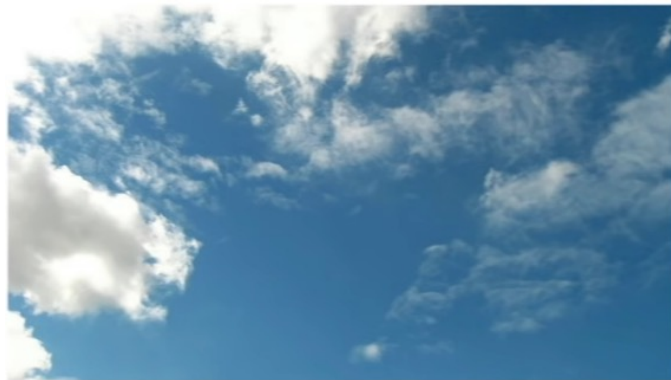


li mousine

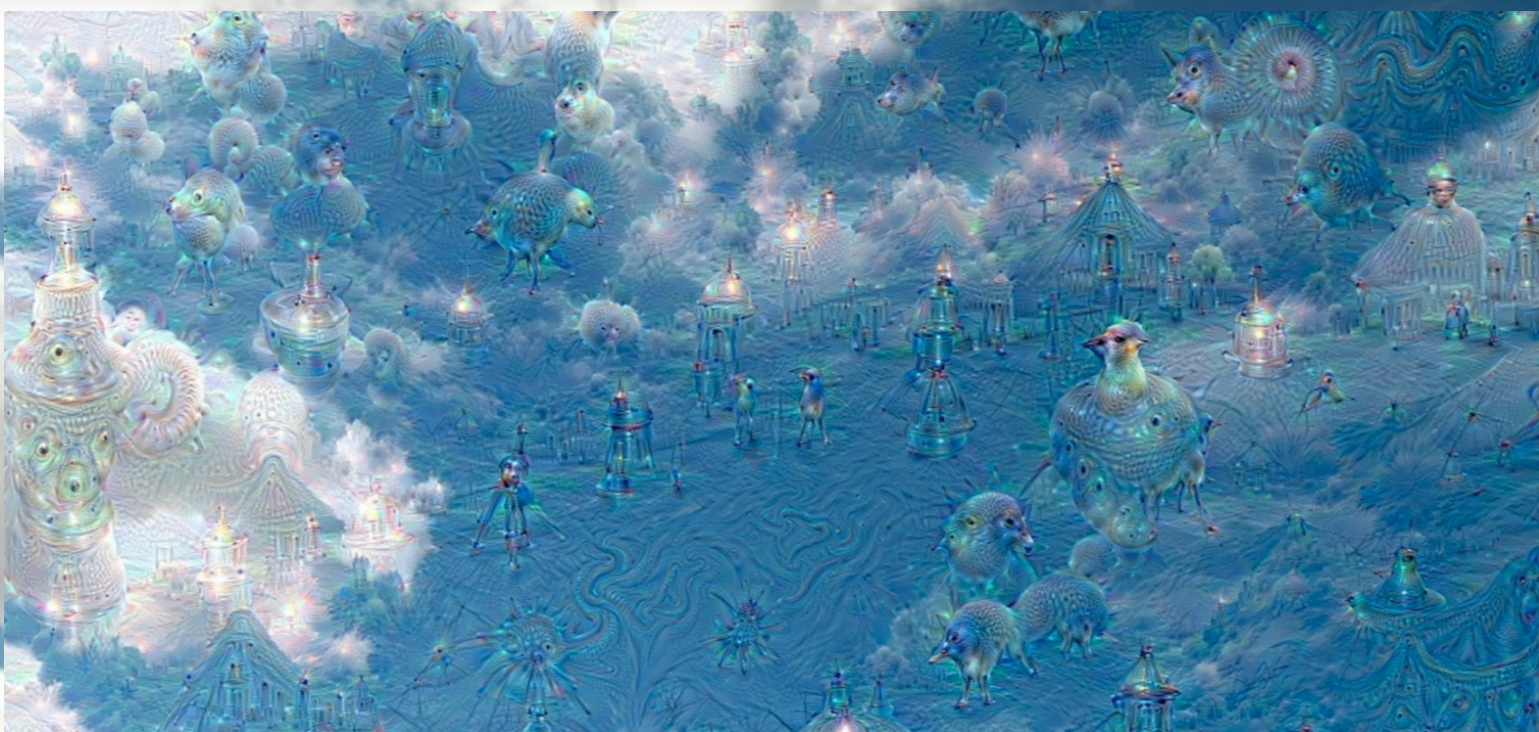
DeepDream

A **computer program** that locates and alters patterns that it identifies in digital pictures.

- Choose an **image I** and a **trained CNN** for classification.
- Let $f_x(I)$ be the activation value of neuron x in layer L when image I is taken in input.
- Change the image so as to maximize $f_x(I)$



- Fix the weights of the trained CNN. Feedforward from input image I to layer L and back-propagate back to I



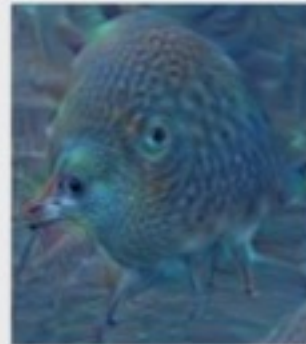
Deep dream trained on ImageNet, where there are many pictures of animals (e.g., dogs and birds), dogs/birds often pop up.



"Admiral Dog!"



"The Pig-Snail"



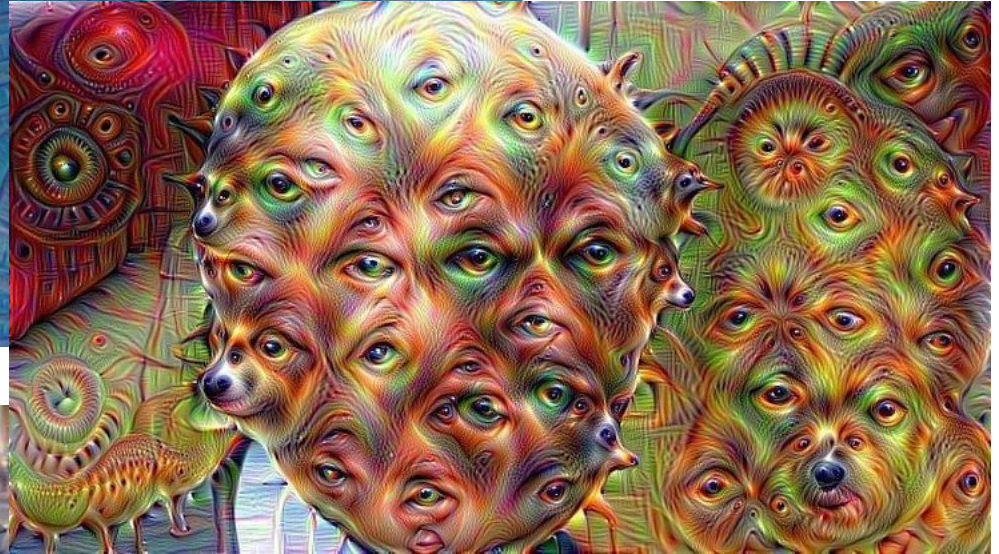
"The Camel-Bird"



"The Dog-Fish"



Lower level
neuron



more
sophisticated

Deep Generative Models

- Deep Dream
- **Style transfer**
- Autoencoder
 - Pretrain network
 - Denoising Autoencoder
- Variation Autoencoder (VAE)
- Generative Adversarial Network (GAN)
 - Conditional-GAN
 - Pix2Pix
 - CycleGAN
- Stable Diffusion

Image Style Transfer

<https://deepdreamgenerator.com/>

Gatys, et al, Image style transfer using convolutional neural networks. 2016.

Take two images.

Extract **content** from an image p , and **style (texture)** from another image a .

Merge them together into one final image x that looks artistic.

Objective:

output image x that has the same content as the input image p but in the style of the style image a

Content Image p



Style Image a



Result Image x

Examples of Same Content with Different Styles

INPUT
IMAGE
 p



OUTPUT IMAGE x



Image Style Transfer

Objective: **output image x** that has the same content as **input image p** but in the style of **style image a**

Find output image that minimizes combination of

→ **content loss** (input image p , output image x)

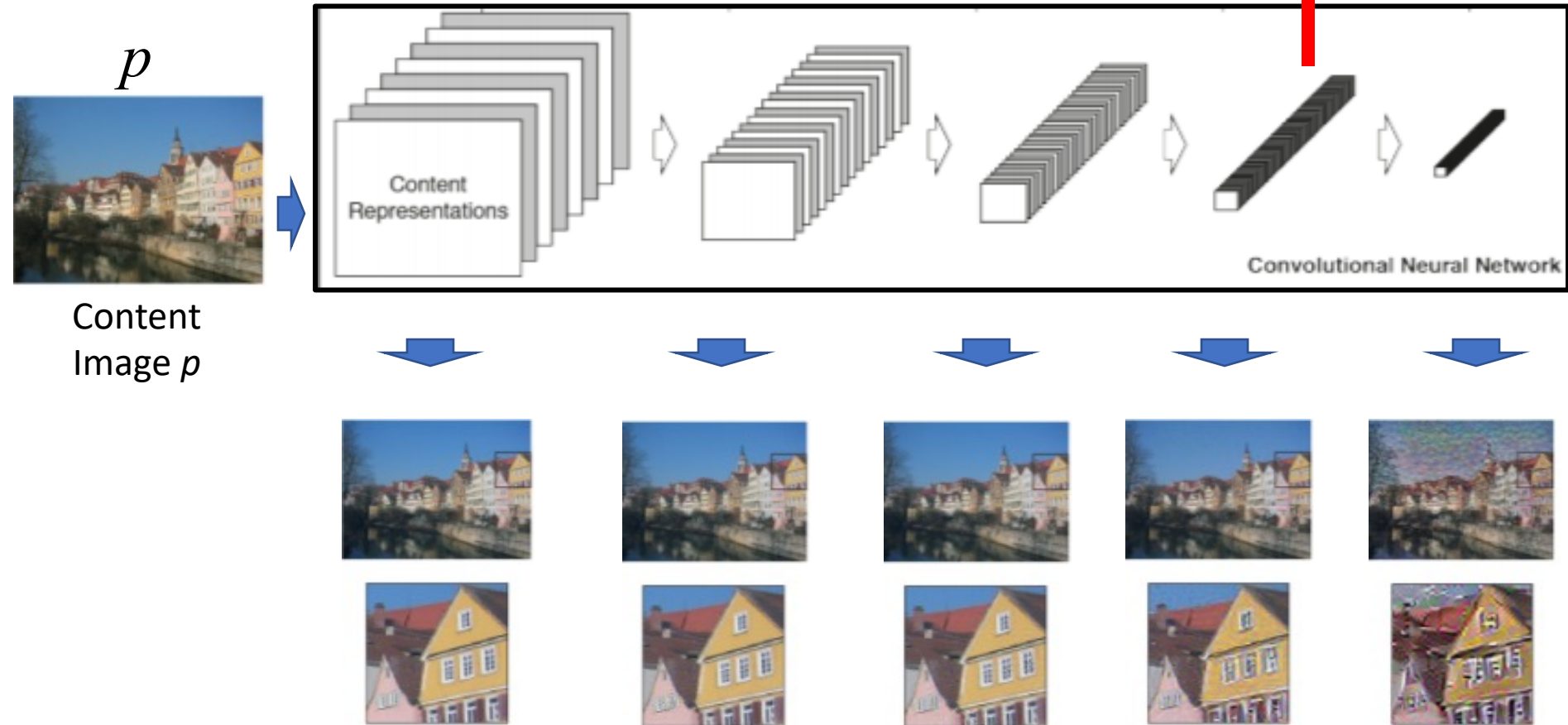
→ **style loss** (style image a , output image x)

Key Observations

- Convolutional neural networks (CNN) can separate the content and the style of images
- *Intermediate layers* of the CNN can capture spatial information about an image

Content of an image

Use output of the convolutional layer P^l as content representation of image p



Minimize Content Loss

Input image contains both content and style (different)



INPUT IMAGE p



OUTPUT IMAGE x

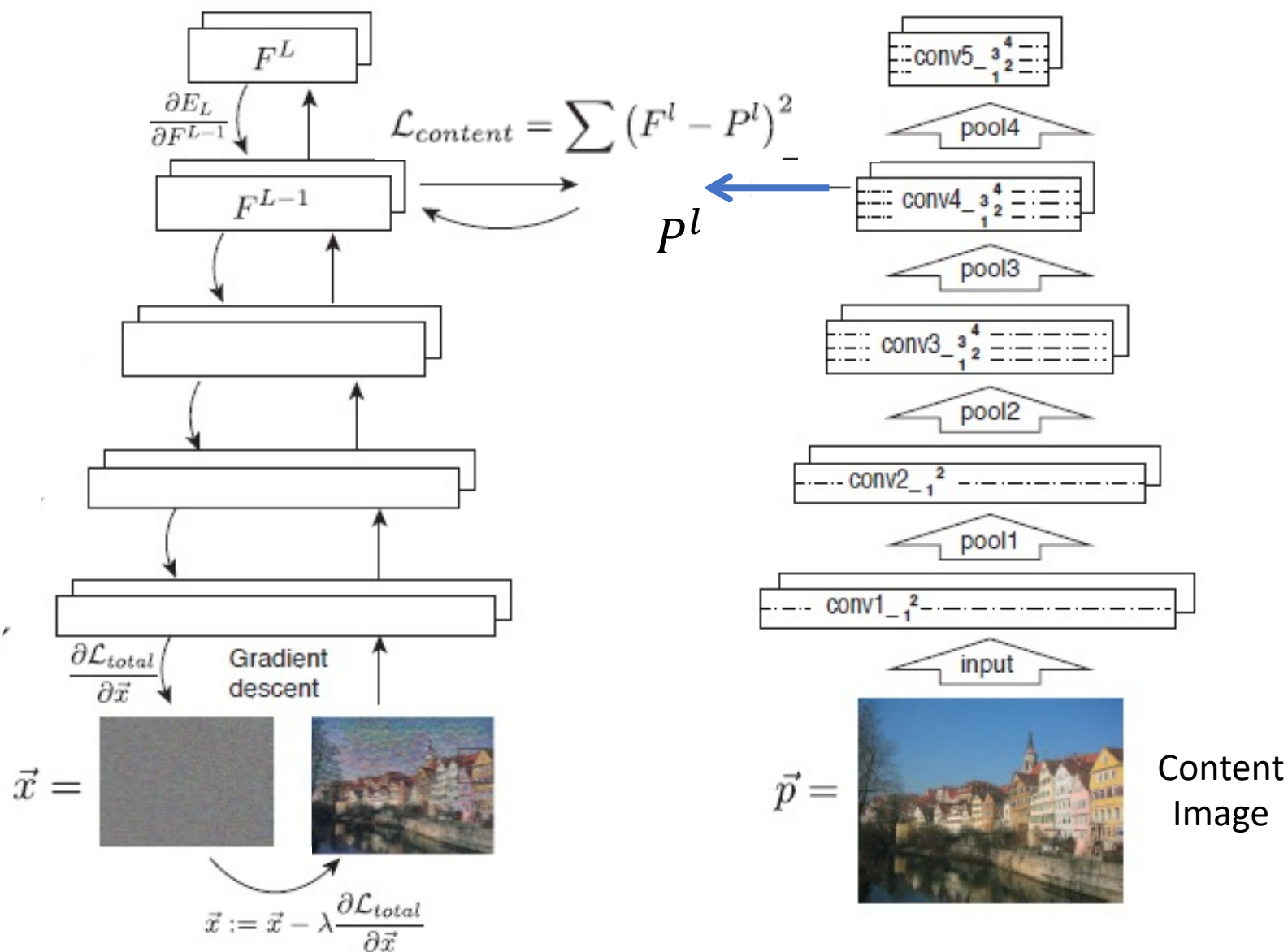
CNN can separate style and content from image

- *MSE loss between initial image x 's features $F(x)$ and content image p 's features $P(p)$ at layer l*

$$L_{content}^l(p, x) = \sum_{i,j} (F_{ij}^l(x) - P_{ij}^l(p))^2$$

- Perform backpropagation to minimize this content loss.
- Modify initial image x until it generates a similar response in a certain layer l as original content image p .

$$\mathcal{L}_{\text{content}} = \sum (F^l - P^l)^2$$



Style Loss

- **What is Style?** *Texture of an image*
- **What is Texture?** *regularity in shape, pattern, colour....*
- *Style as texture* is encapsulated by **Gram Matrix**
Features which are *correlation features* extracted from many layers of a pre-trained CNN
- **Gram Matrix** (size $N \times N$, for N feature maps of size $M \times M$)

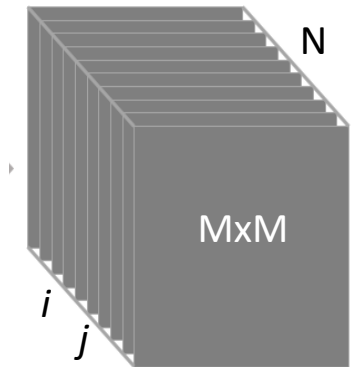
$$G_{ij}^l = \sum_k F_{ik}^l F_{jk}^l. \quad \text{Dot product of features } i \text{ and } j$$

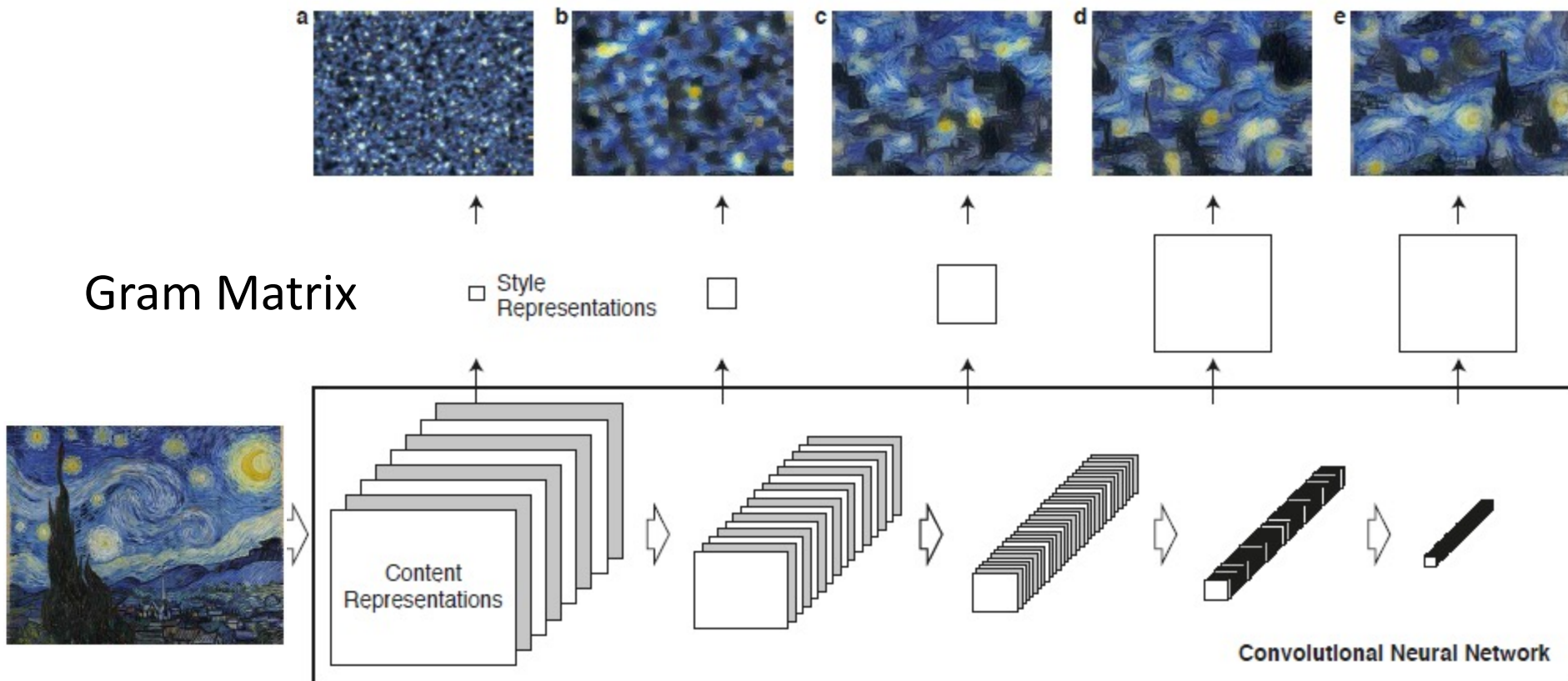
Dot product describes the relation between features

- A^l - Gram Matrix in layer l of style image a

$$E_l = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} (G_{ij}^l - A_{ij}^l)^2$$

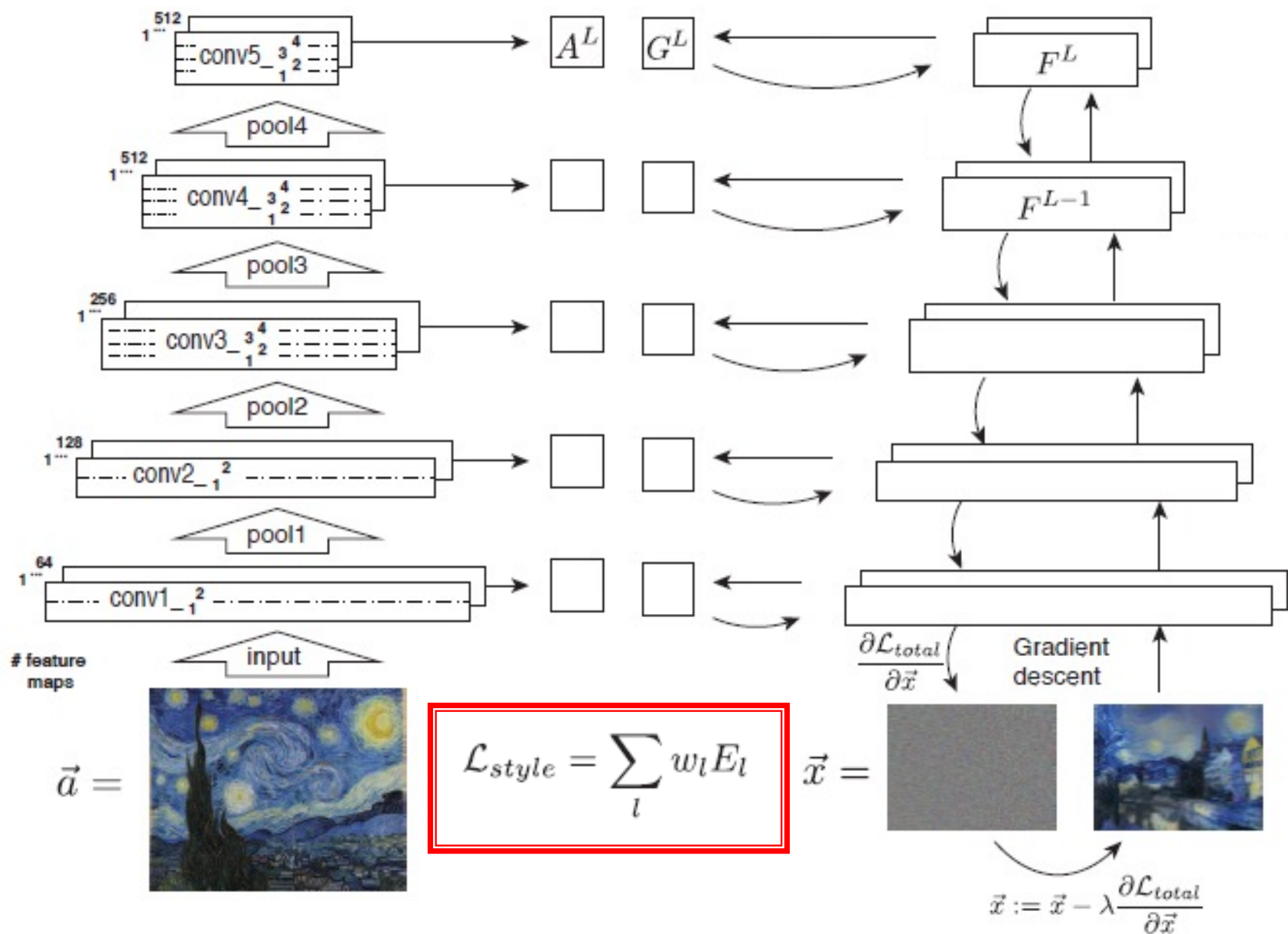
- **Style Loss** $\mathcal{L}_{style} = \sum_l w_l E_l$





Style representation computes correlations
between
different features in each layer

$$E_L = \sum (G^L - A^L)^2$$



Loss Function

- Total style loss across each layer is

$$\mathcal{L}_{\text{style}}(\vec{a}, \vec{x}) = \sum_{l=0}^L w_l E_l$$

- Weighted sum of content and style loss to minimize

$$\mathcal{L}_{\text{total}}(\vec{p}, \vec{a}, \vec{x}) = \alpha \mathcal{L}_{\text{content}}(\vec{p}, \vec{x}) + \beta \mathcal{L}_{\text{style}}(\vec{a}, \vec{x})$$

where $\vec{p}$ is the content image

$\vec{a}$ is the style image

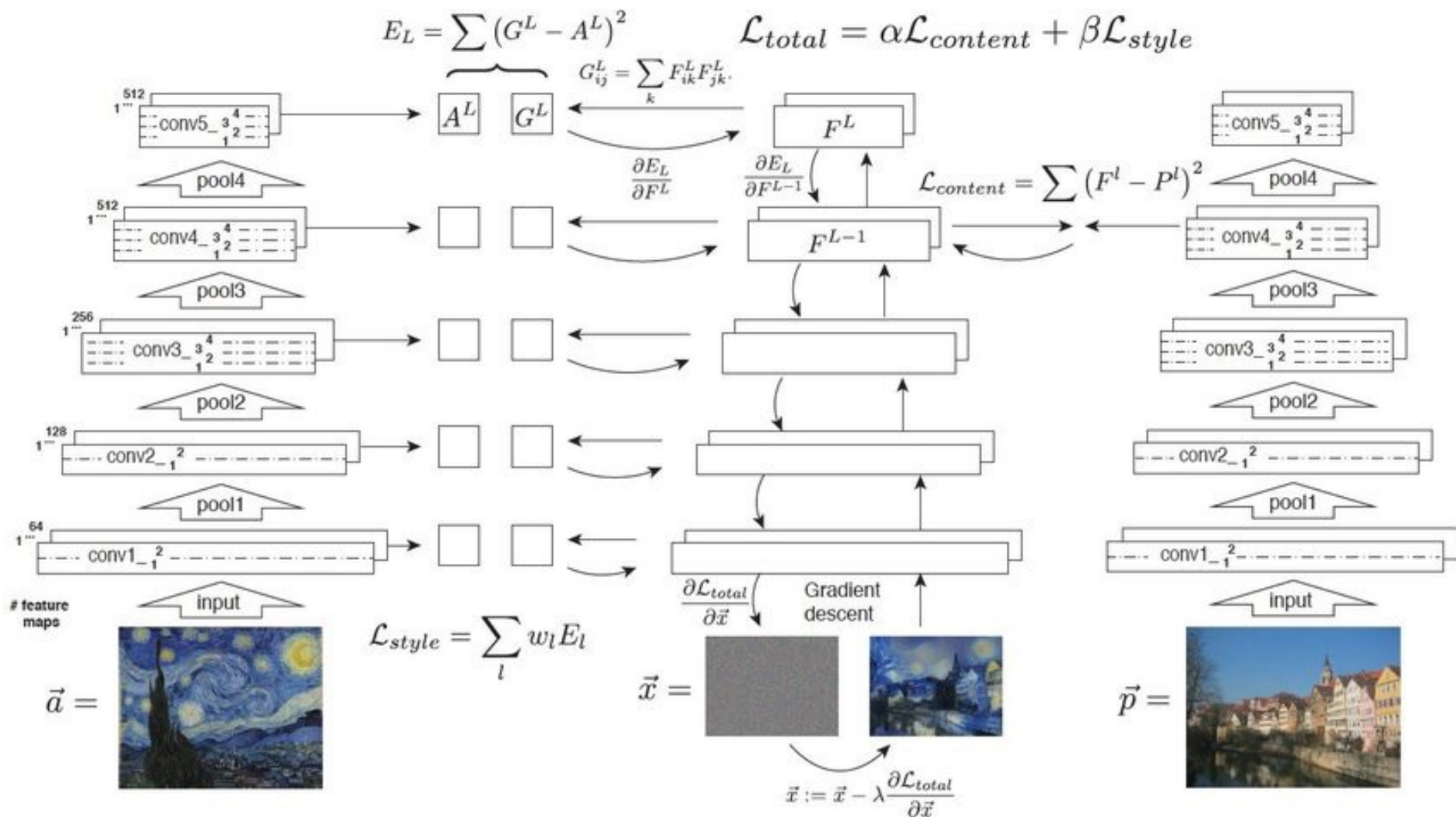
$\vec{x}$ is the input image

α and β are weighting factors

for content and style respectively.

- The gradient during the optimization is calculated w.r.t. pixels.

Style transfer Process



Some examples depicting the effects of varying the ratio between α / β :
(larger ratio implies more content)



Spatial Control in Neural Style Transfer

L.A. Gatys, et al, Controlling Perceptual Factors in Neural Style Transfer, 2017



(a) Content



(b) Style I



(c) Style II



Color Control in Neural Style Transfer

Preserve
colors of
original
content image



(a) Content

Perform
style transfer
without
transferring
color from
style image
onto the
content image



(b) Style



(c) Without color control



luminance-only style transfer



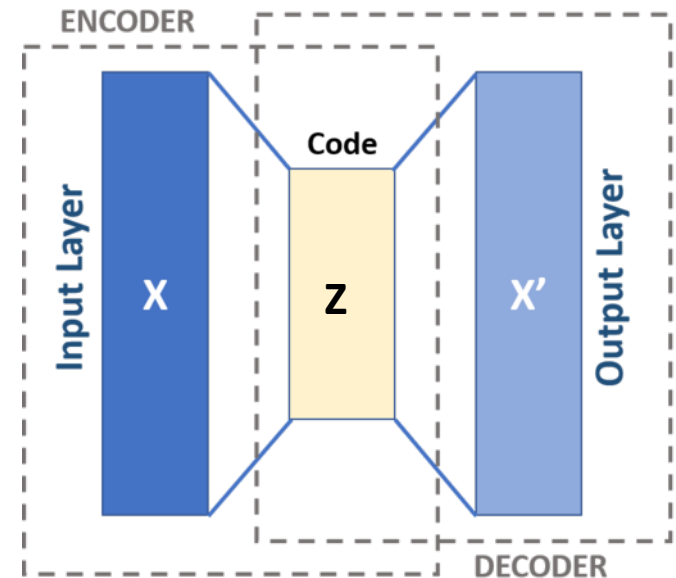
colour histogram matching

Deep Generative Models

- Deep Dream
- Style transfer
- **Autoencoder**
- Variation Autoencoder (VAE)
- Generative Adversarial Network (GAN)
 - Conditional-GAN
 - Pix2Pix
 - CycleGAN
- Stable Diffusion

Autoencoder

- **Autoencoder** has an input layer, hidden layer, and an output layer.
- The input is forced to be identical to the output, so it is the **hidden layer** we are interested in.
- The hidden layer forms a kind of **encoding** of the input.
- Autoencoder learns efficient data coding in an **unsupervised** manner.
- The aim of an autoencoder is to learn a representation (encoding) for a set of data, typically for **dimensionality reduction**.



Autoencoders

- **Encode:** $\mathbf{x} \rightarrow \mathbf{z}$
 $\mathbf{z}$ of lower dimensionality than $\mathbf{x}$
- **Decode:** $\mathbf{z} \rightarrow \mathbf{x}'$ where $\mathbf{x}' \approx \mathbf{x}$
- One way to map:

$$\mathbf{z} = g(W_1\mathbf{x} + b_1)$$

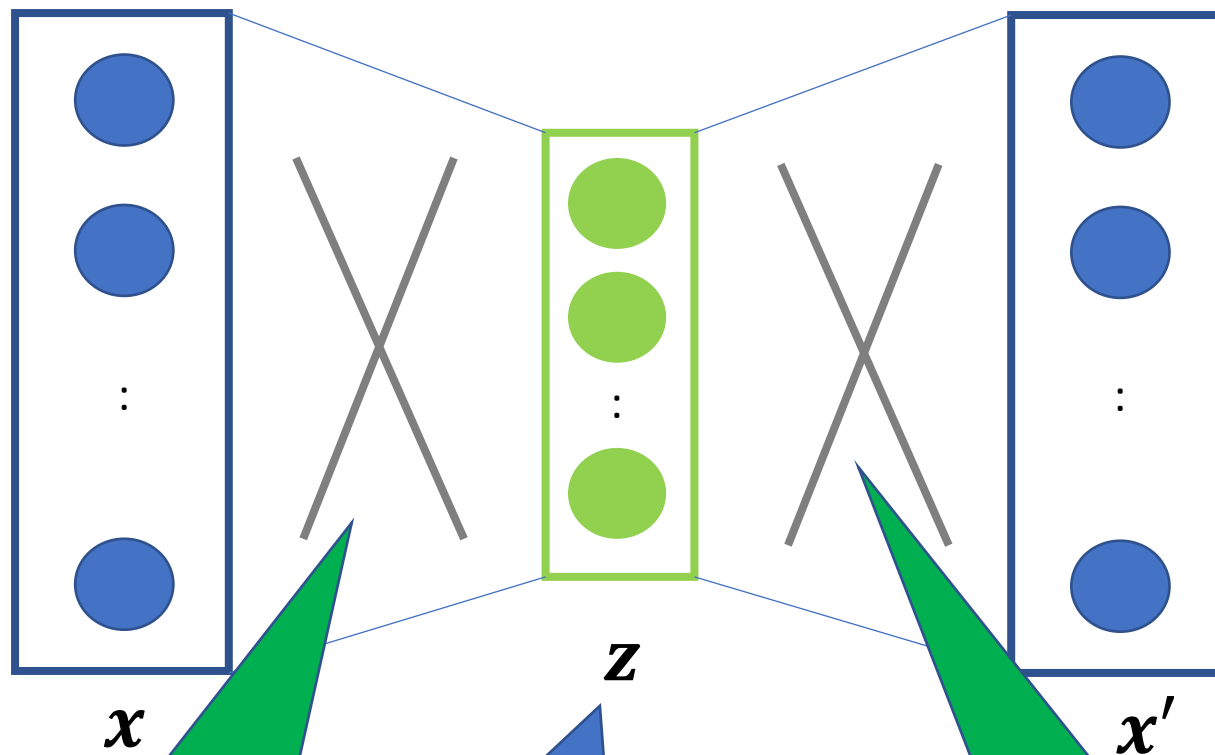
$$\mathbf{x}' = g(W_2\mathbf{z} + b_2)$$

$$\text{Minimize } J(W_1, b_1, W_2, b_2) = \sum_{i=1}^m (\mathbf{x}' - \mathbf{x})^2$$

Reconstruction
loss

Autoencoder

Encoder – Decoder Model

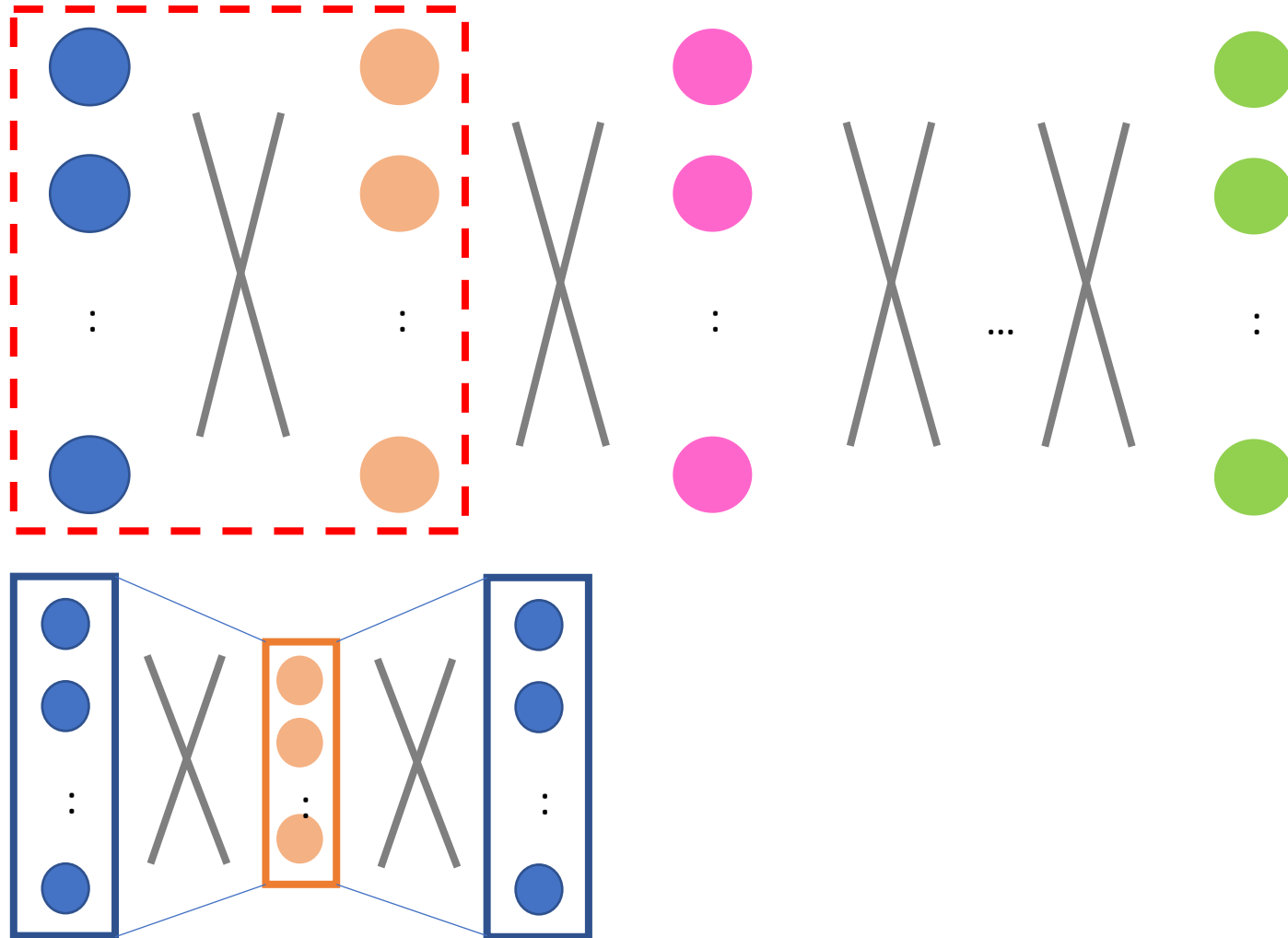


Encoder preserves relevant and discards irrelevant information in limited encoding

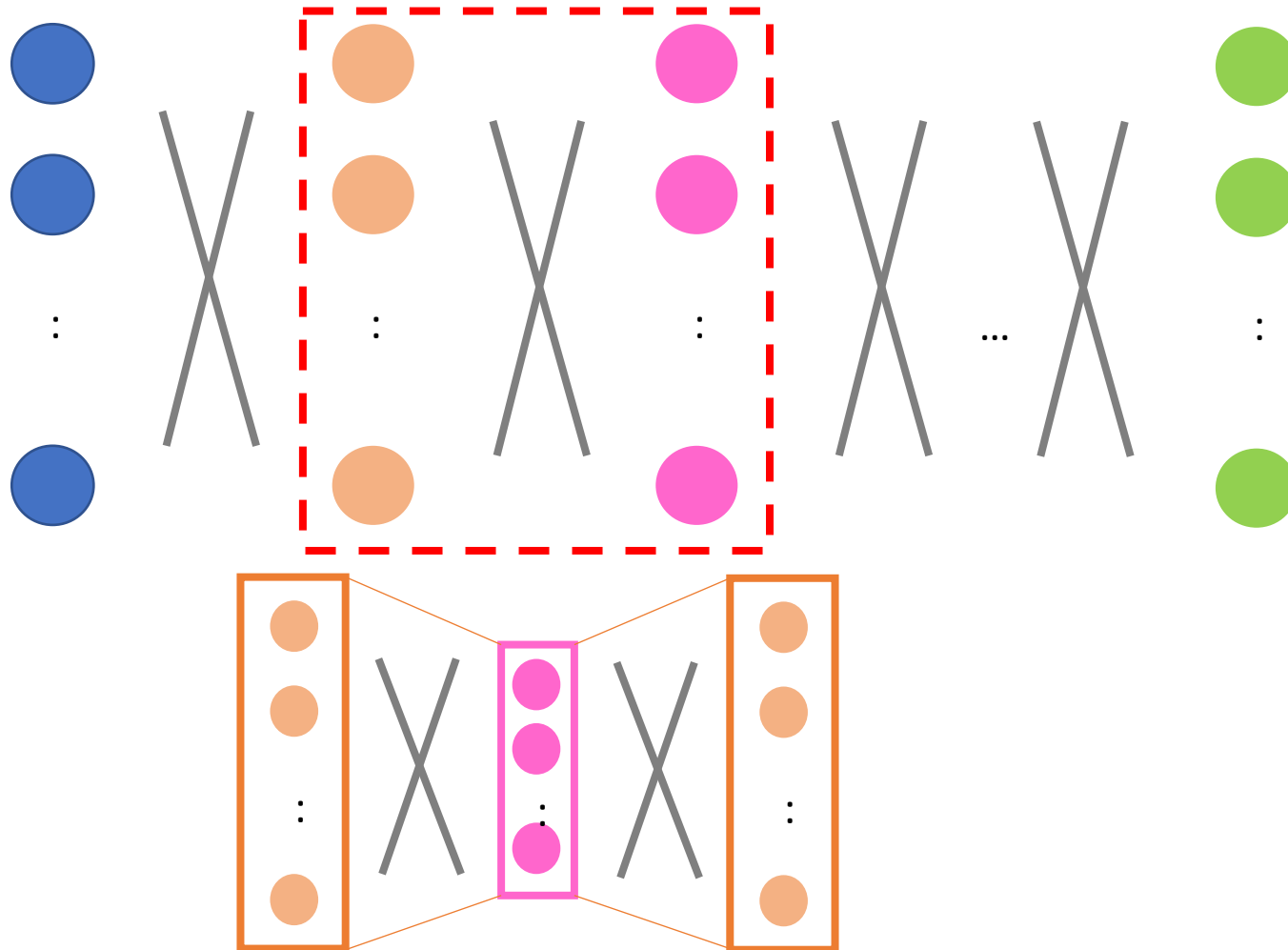
Far less units than input (encoding)
Latent variables

Decoder takes the encoding and reconstructs the full input.

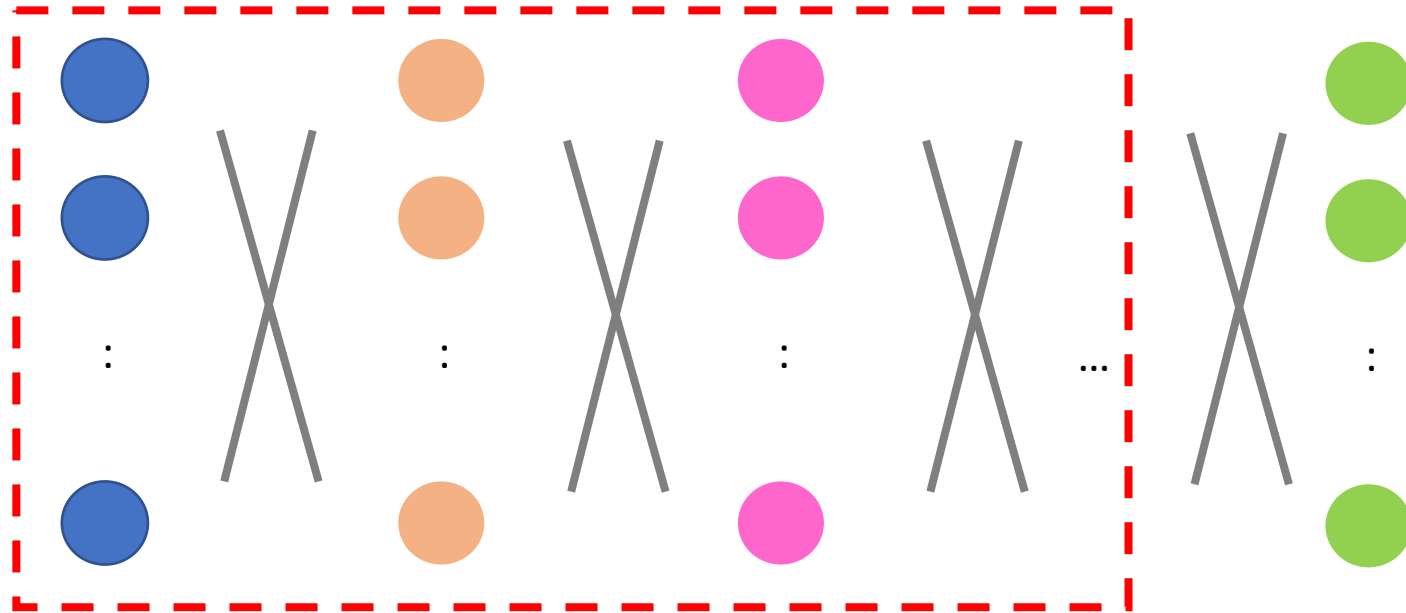
1. Using **autoencoders** to pre-train networks



Using **autoencoders** to pre-train networks

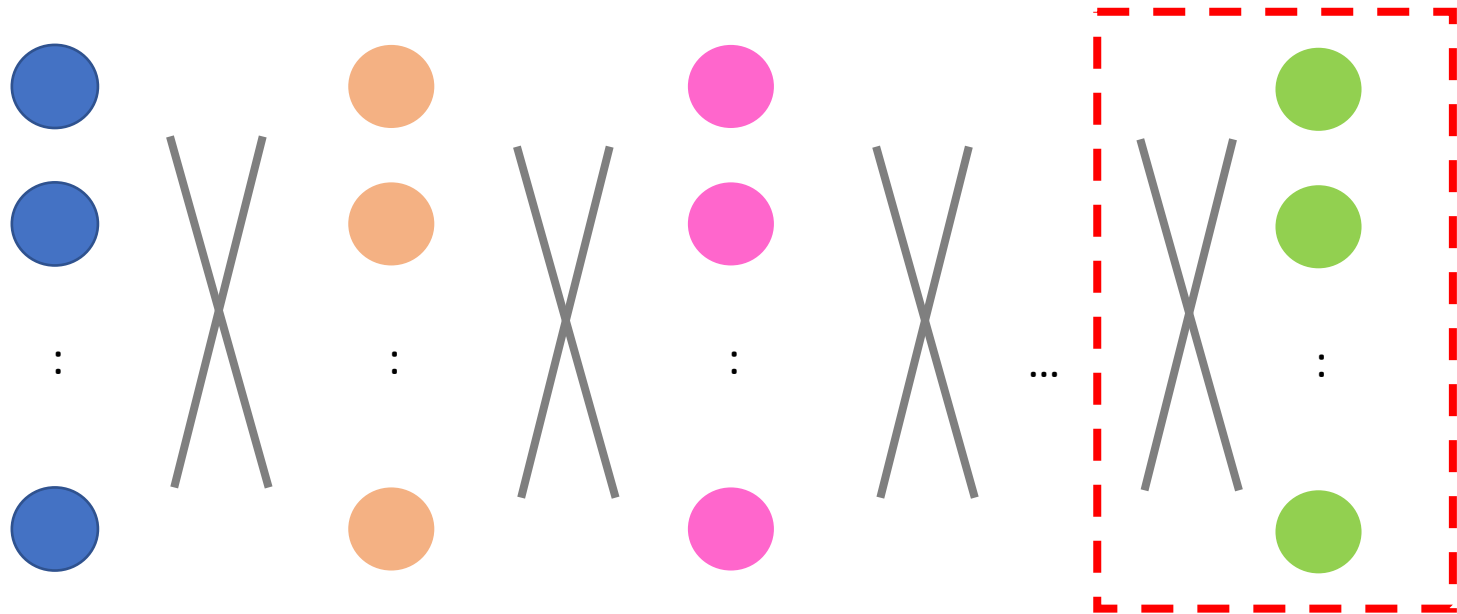


Using **autoencoders** to pre-train networks



Pre-training step: train a sequence of shallow autoencoders, greedily one layer at a time, using unsupervised data

After pre-training, train with supervised data



Fine-tuning step 1: train last layer using supervised data

Fine-tuning step 2: train entire network using supervised data

2. Recovery from Corrupted Images

Problem:

- How to recover corrupted images?

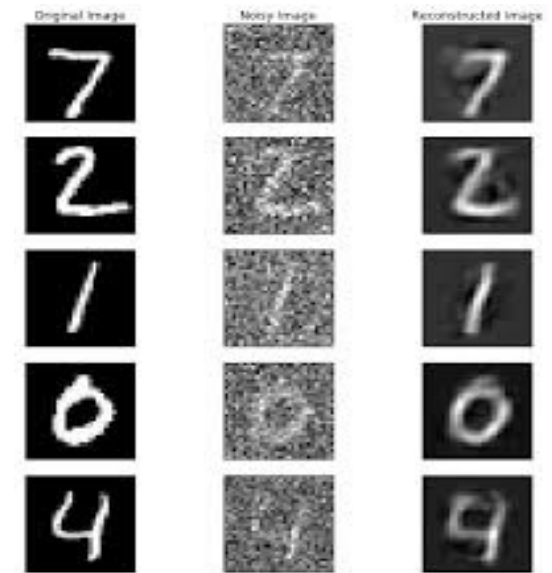
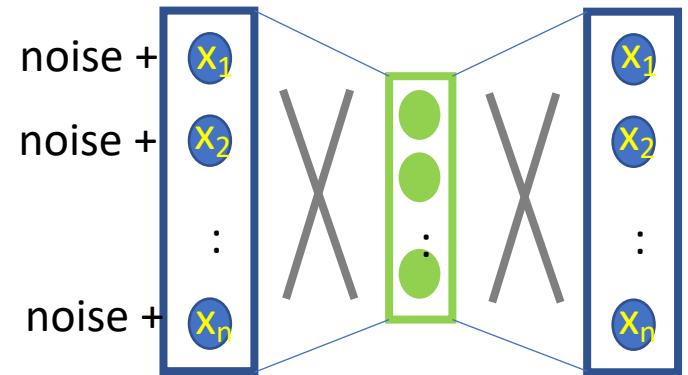


- Can we train an NN to recover the proper images?



Denoising Autoencoder (DAE)

- DAEs recover the original input from **corrupted input**, i.e., cleaning or *denoising*
- DAEs solve this problem by adding noises on purpose to the original input and randomly turning some of the input values to zero.
- The cost function is to compare the output values with the original input, not with the corrupted input.

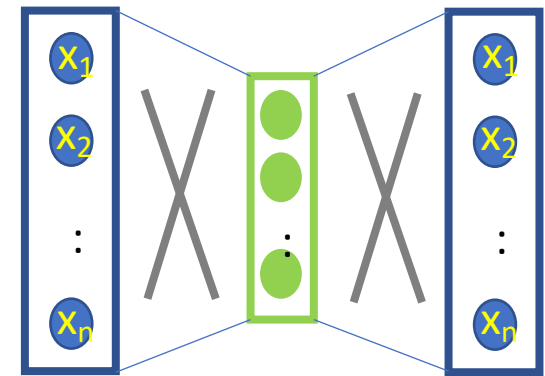


original corrupted denoised

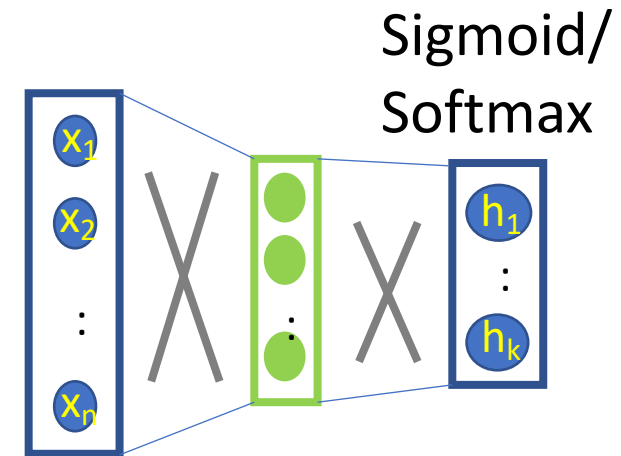
3. Use Autoencoder as Classifier

Any autoencoder can be turned to a classifier.

- Train an autoencoder for the data to be classified (the hidden layer contains latent variables, important features)
- Remove the decoder part and replace it by a Sigmoid (binary) or Softmax (multi-class) output layer.
- Train the network for a second time by tuning only the weights in the output layer (keep all the encoder weights frozen).
- A classifier that only uses a small set of features. Autoencoder is used as a feature extractor.



Autoencoder



Sigmoid/
Softmax

Classifier

Sparse Autoencoder (SAE)

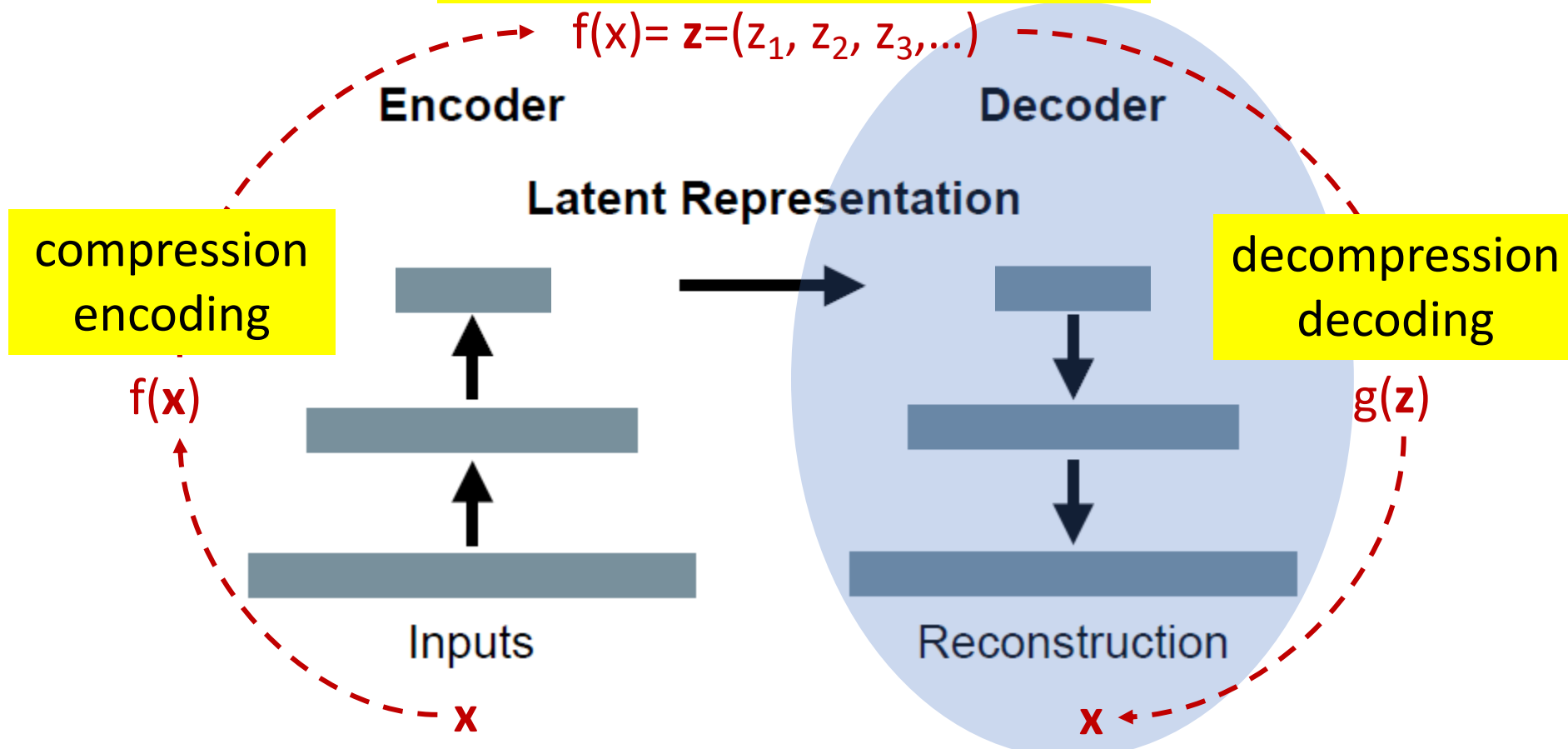
- As humans, our brains activate only a small subset of neurons in response to specific stimuli.
- A **sparsity constraint** on the hidden layer activations encourages only a small number of neurons in the hidden layer to be active.
- Enforce sparsity through regularization, such as L1, i.e., $L = ||X' - X||^2 + \lambda \cdot \text{penalty}(z)$
- Sparsity prevents Autoencoder from Overfitting –
 - Feature extraction: learn the most critical features
 - Dimensionality reduction: lower-dimension representation
 - Interpretation: know features (internal neuron activations)
- **SAE** extracts interpretable features, enhances model robustness, and ensures efficiency.

Deep Generative Models

- Deep Dream
- Style transfer
- Autoencoder
- **Variation Autoencoder (VAE)**
- Generative Adversarial Network (GAN)
 - Conditional-GAN
 - Pix2Pix
 - CycleGAN
- Stable Diffusion

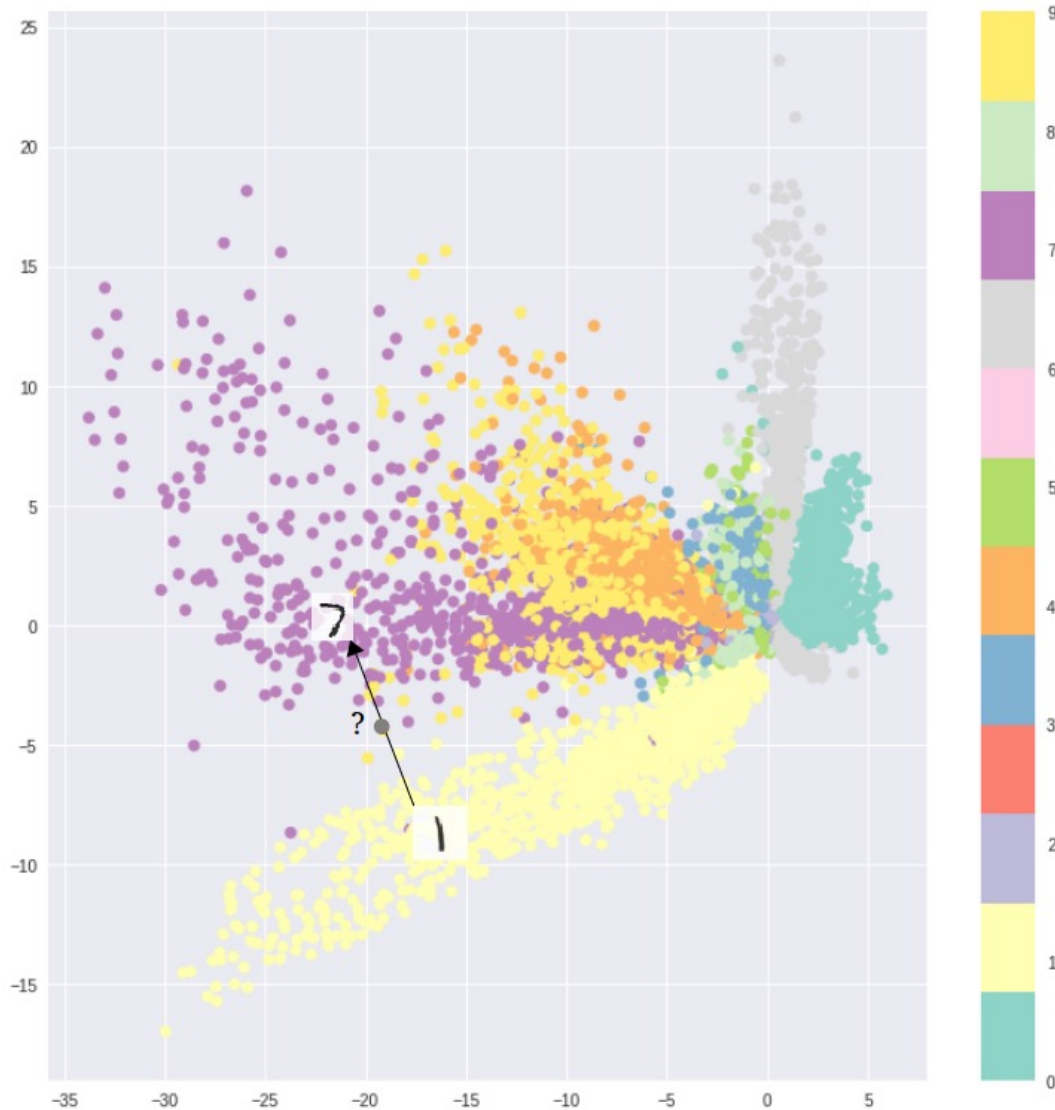
Recall: Autoencoders

Latent variable z contains fundamental information of x .



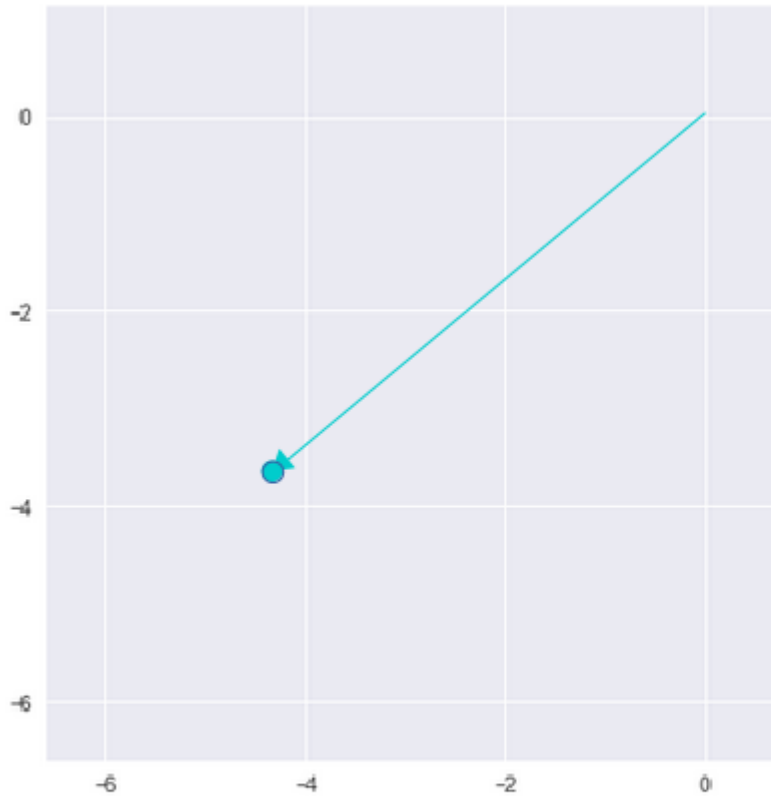
Images are produced based on the latent variables z
Different latent variables will produce different images.

Problem with standard autoencoders

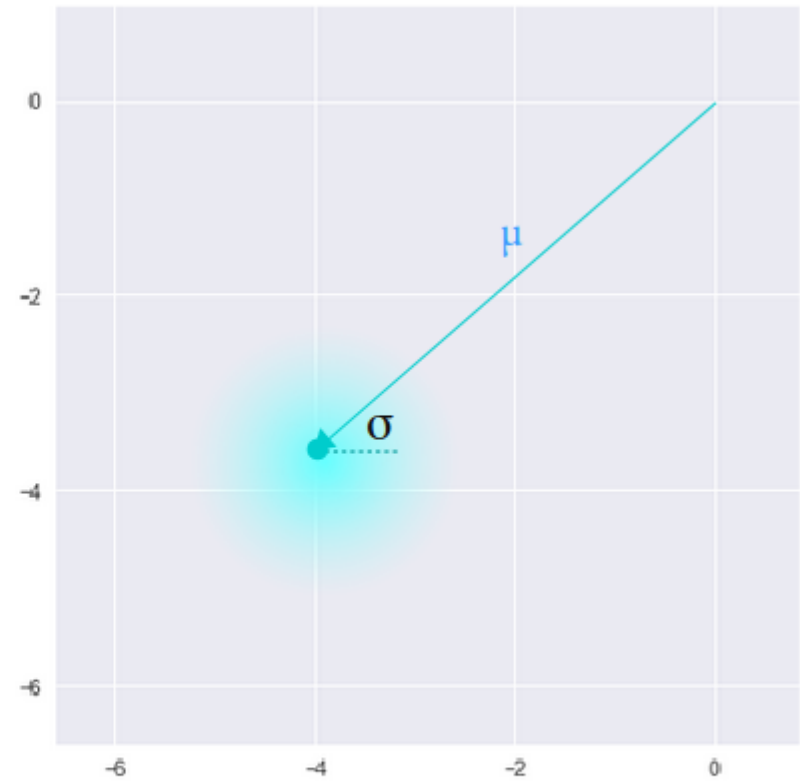


Distinct clusters →
good for decoding
and *replicating* the
same image

**Gaps between
clusters
(or discontinuities)**
→ not so good for
generating images,
i.e., not meaningful
images

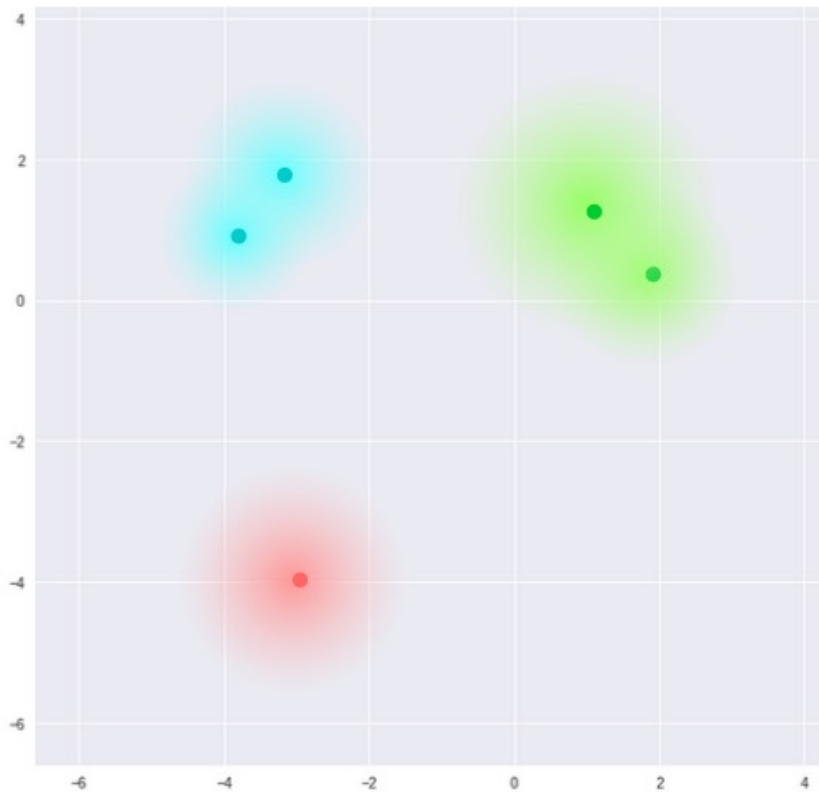


Standard Autoencoder
(direct encoding coordinates)

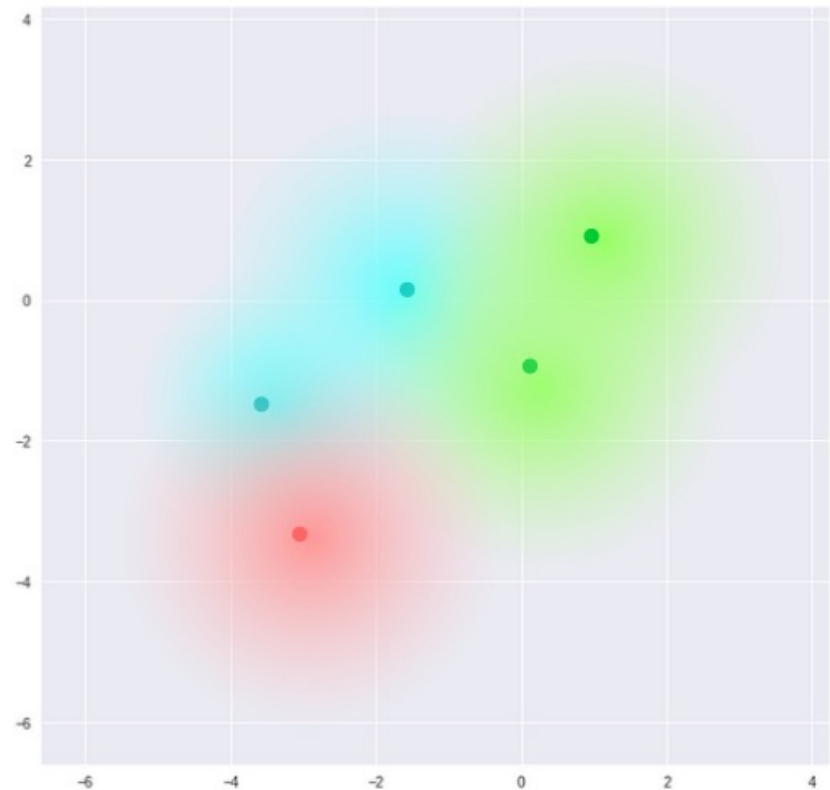


Variational Autoencoder
(μ and σ initialize a probability distribution)

Latent variables which produce a particular image.



What we may inadvertently end up with



What we require

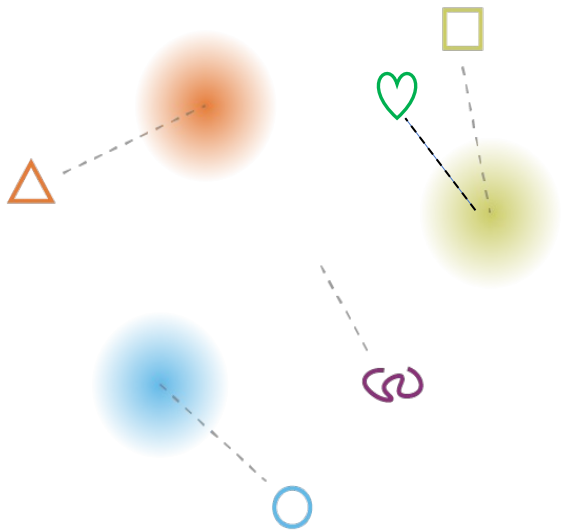
Want encodings to be as close as possible to each other (while still being distinct), allowing smooth interpolation and enabling the construction of new samples.

VAE Latent Space

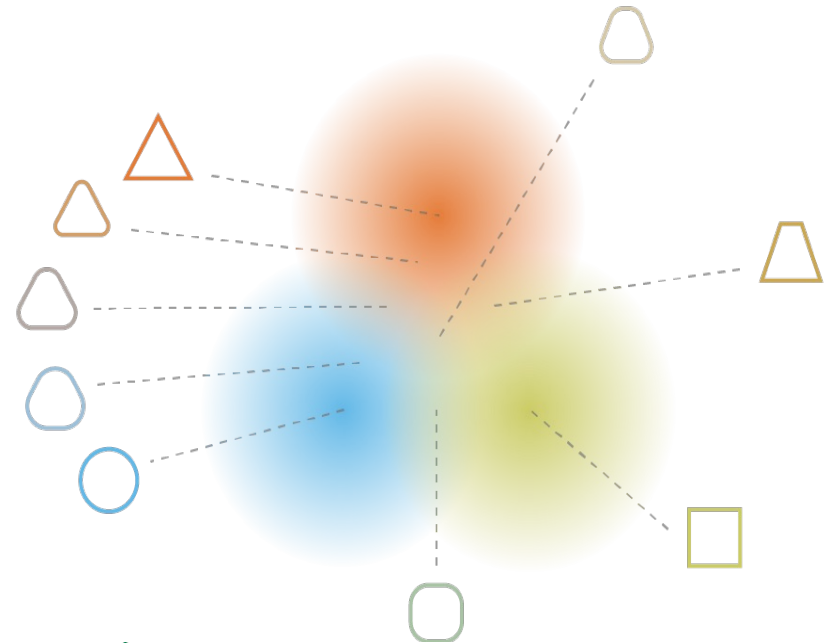
Two main properties:

Continuity (two close points in the latent space should not give two completely different contents once decoded)

Completeness (for a chosen distribution, a point sampled from the latent space should give “meaningful” content once decoded).



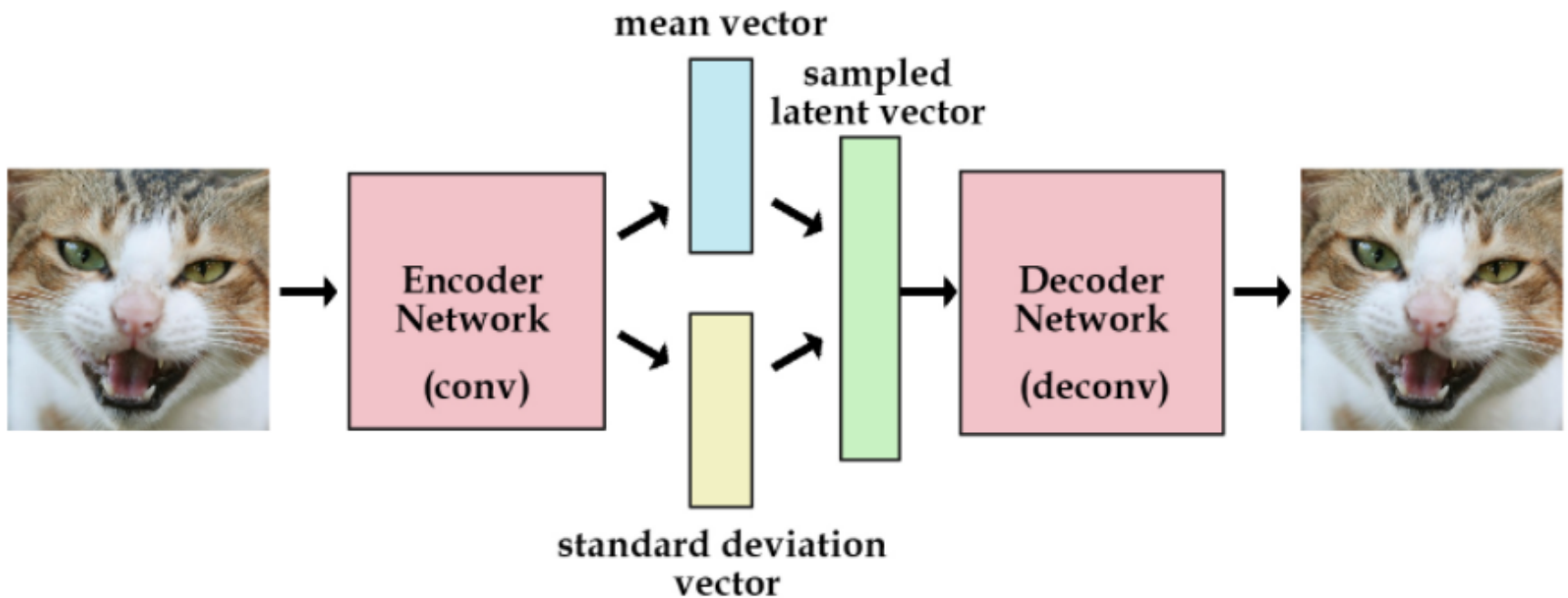
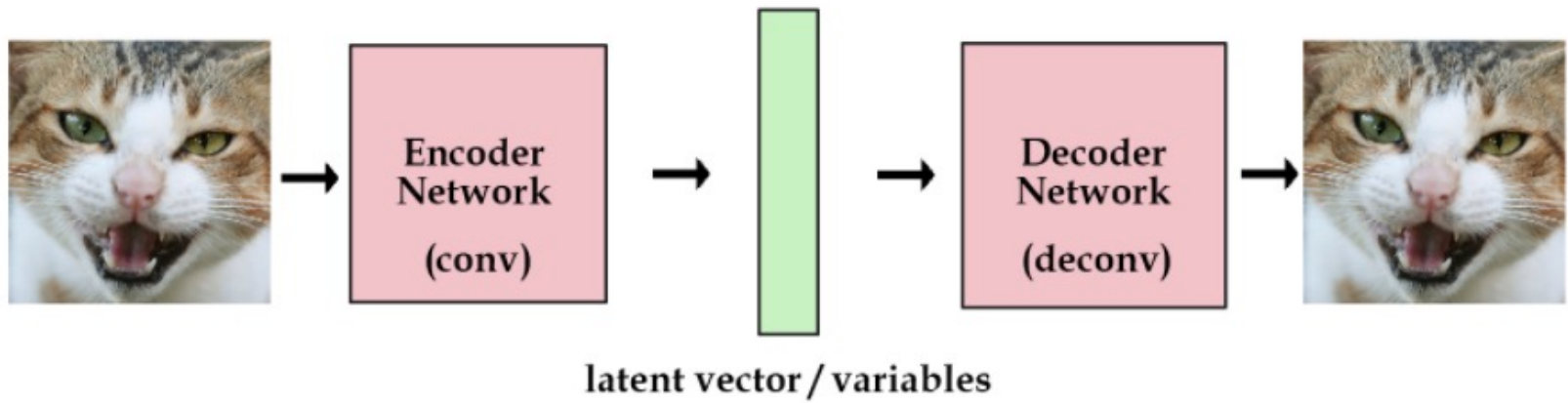
what can happen without regularisation



what we want to obtain with regularisation

Variational Autoencoders (VAEs)

- Generative **extension of autoencoders** that **constrains the output of the encoder**
- By design, their latent spaces (i.e., the space for their **latent representation vectors**) are continuous, allowing easy random sampling and interpolation
- Encoder (instead of outputting one encoding vector of size n) outputs **two** vectors of size n :
a **mean vector μ** , and a **standard deviation vector σ**
- Difference between autoencoder and VAE
 - AEs learn a “compressed representation” of input
 - VAEs learn the “model”, i.e., parameters of a probability distribution representing the input. VAEs can sample from the distribution and generate new input data samples.



Loss Function for VAE

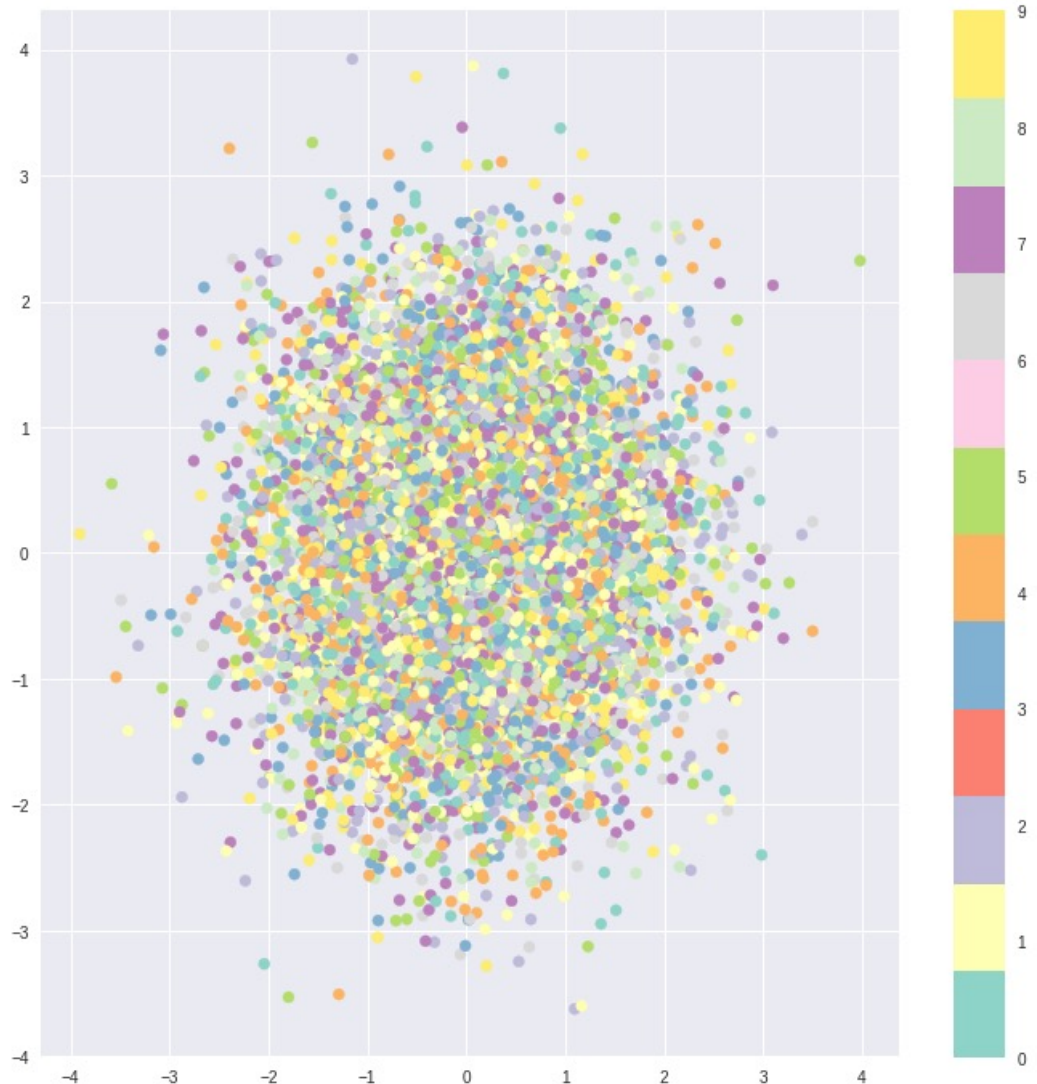
- Objective to minimize

$$\begin{aligned}\text{Loss} &= \text{reconstruction loss} + \text{KL divergence loss} \\ &= -E_z (\log p(x|z)) + D(q, p)\end{aligned}$$

- Introduce **Kullback–Leibler (KL)** divergence loss into the loss function as the regularization term
 - a measure of how much a model probability distribution Q is different from a true probability distribution P .
- Denoted as **$D(q, p)$** which describes the divergence between the probability distributions p and q and tries to move the distributions closer

Minimizing KL divergence loss alone

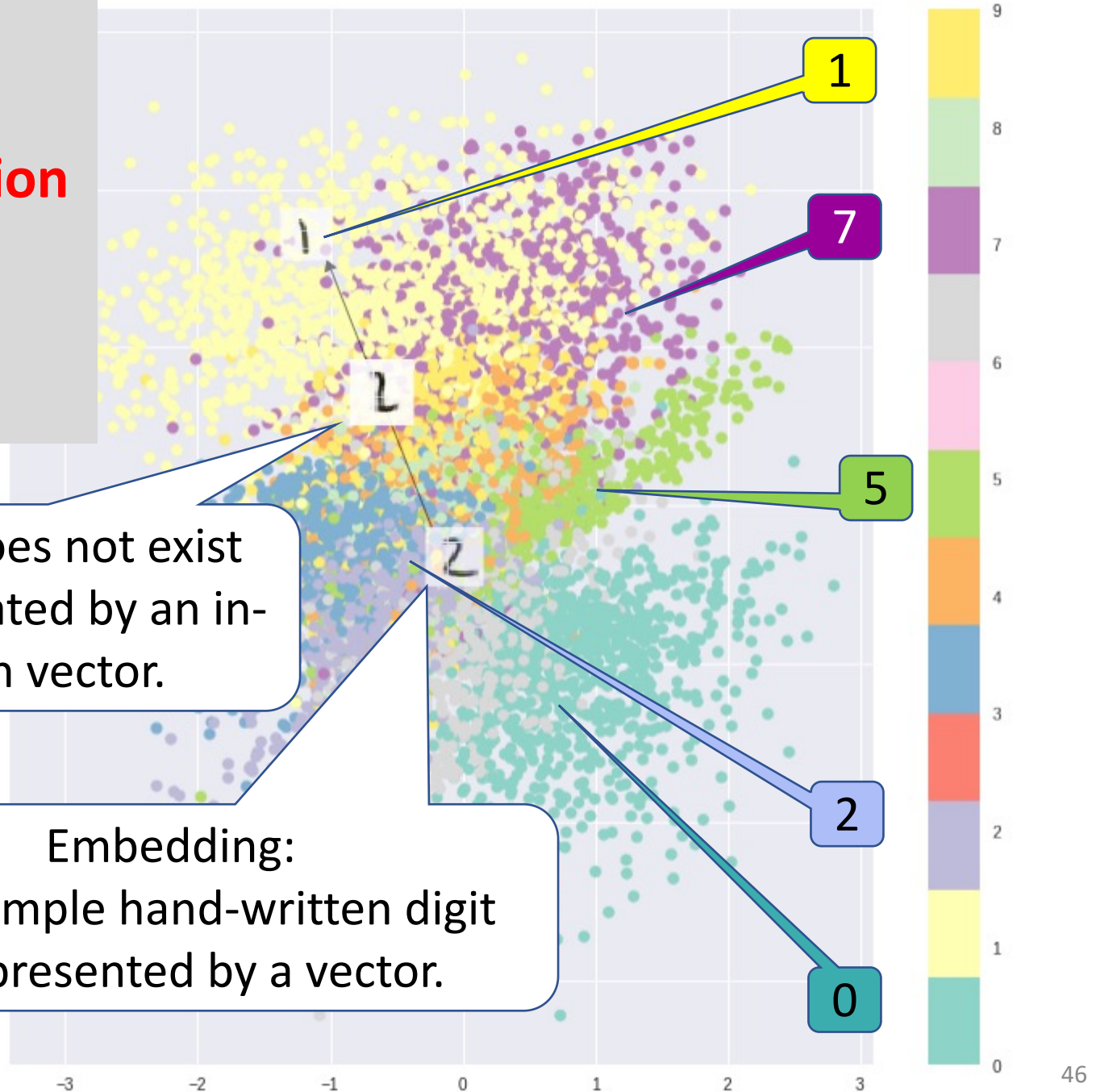
- Minimizing KL loss encourages the encoder to distribute all encodings evenly around the center of the latent space
- Just minimizing KL loss alone (see figure at the right)



Minimizing
both
**reconstruction
loss** and **KL
divergence
loss**

This digit does not exist
and is generated by an in-
between vector.

Embedding:
each sample hand-written digit
is represented by a vector.



Deep Generative Models

- Deep Dream
- Style transfer
- Autoencoder
- Variation Autoencoder (VAE)
- **Generative Adversarial Network (GAN)**
 - Conditional-GAN
 - Pix2Pix
 - CycleGAN
- Stable Diffusion

Generative Adversarial Network (GAN)

- training process as a game
between two separate networks

Generative Adversarial Network

VAE is pretty bad for generating new data, because VAE learns an “averaged” representation of the data (blurry output)

- **G**enerative • Learn a generative model
- **A**dversarial • Trained in an adversarial setting
- **N**etworks • Use Deep Neural Networks

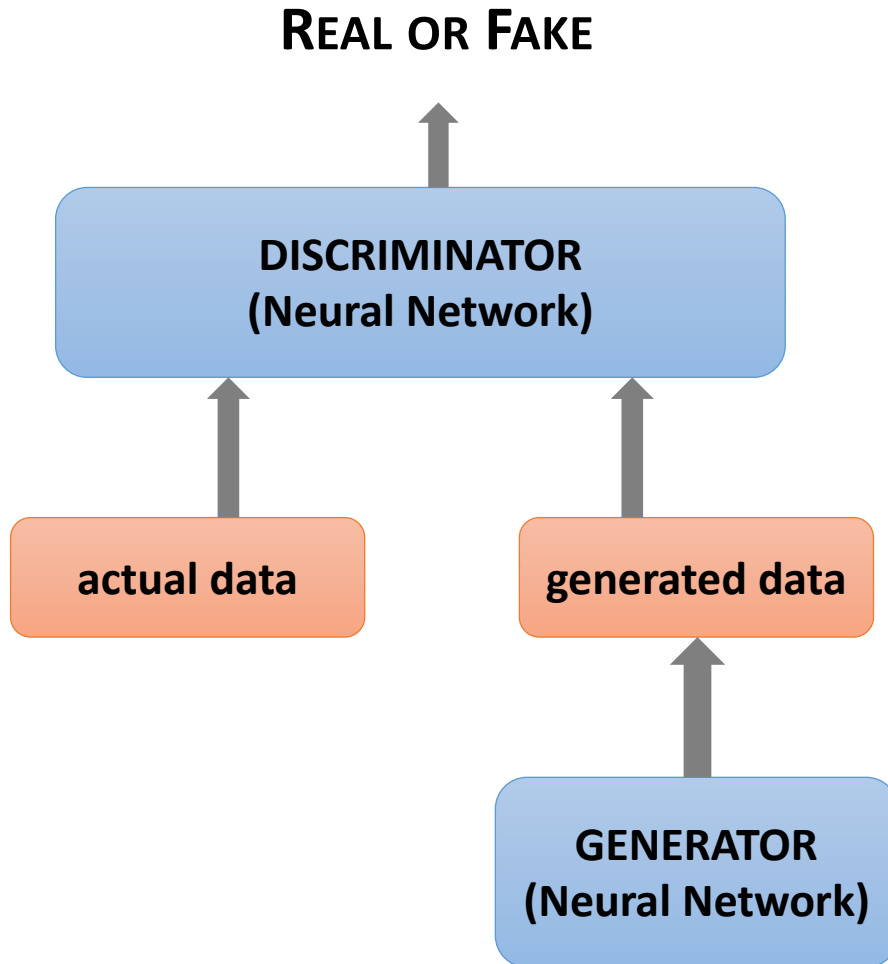
Which one is Computer generated?



computer
generated



Generative Adversarial Networks (GANs)



Training is a game

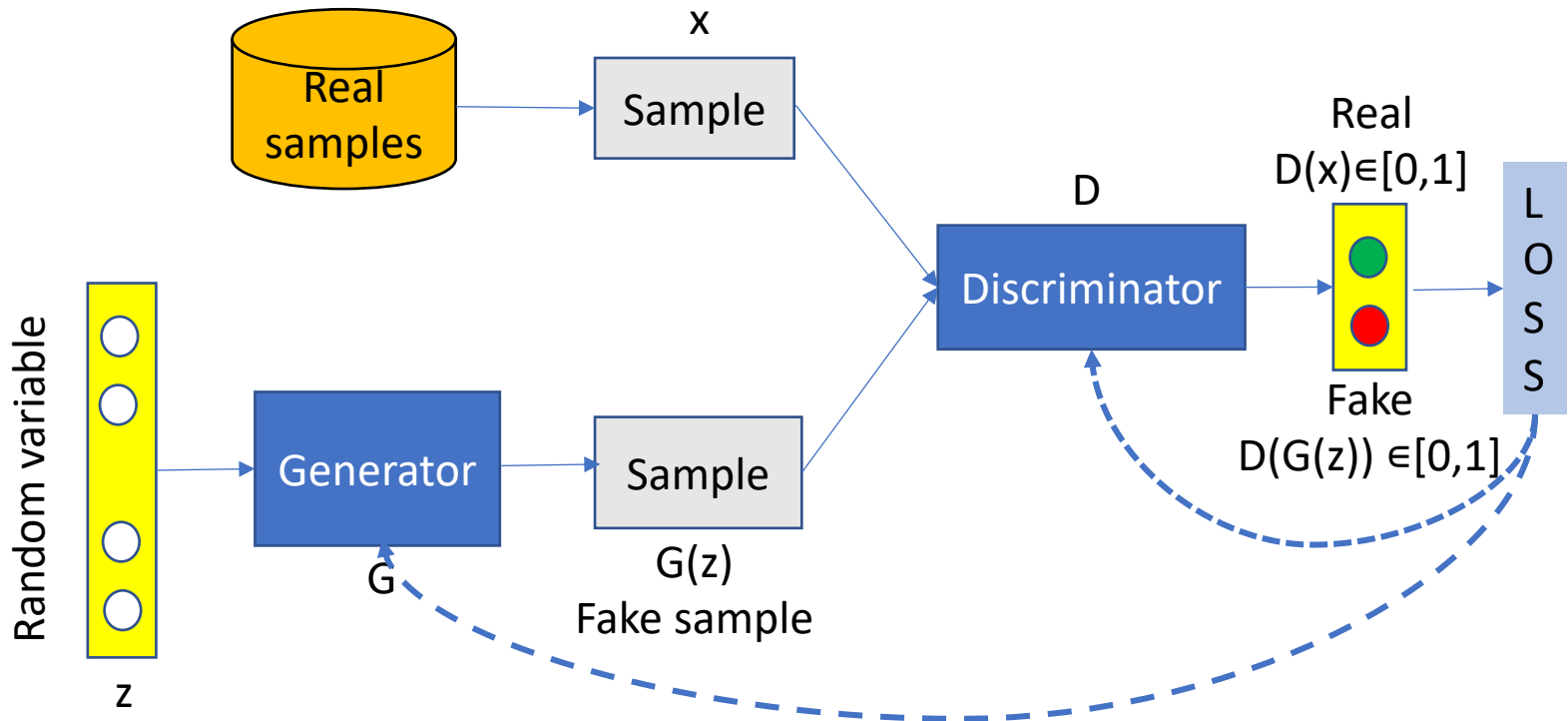
between two separate networks:

a **generator** network and a **discriminator** network.

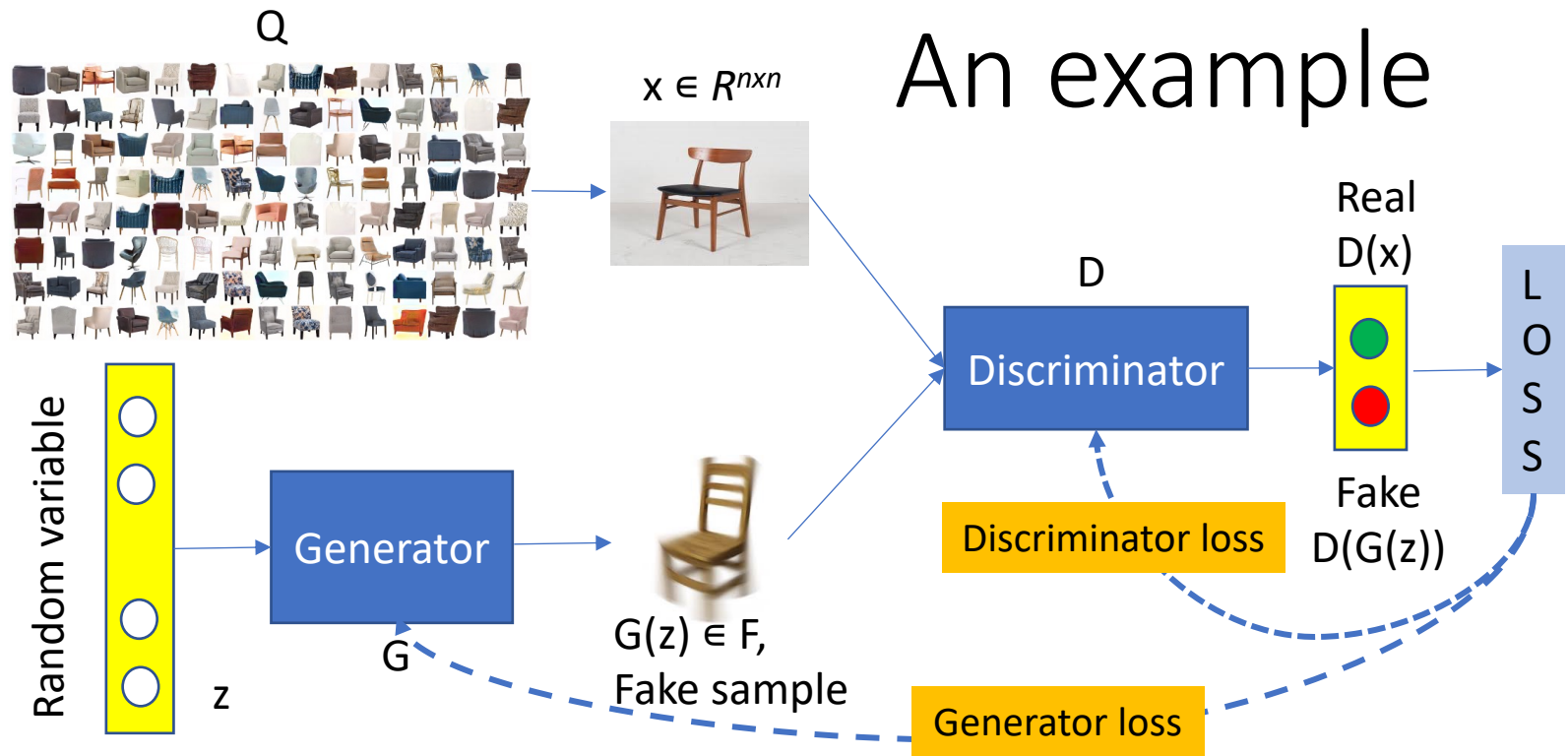
Discriminator network tries to classify samples as coming from the true distribution R , or the generator's model distribution F .

Generator network tries to generate samples to fool the discriminator.

Generative Adversarial Network

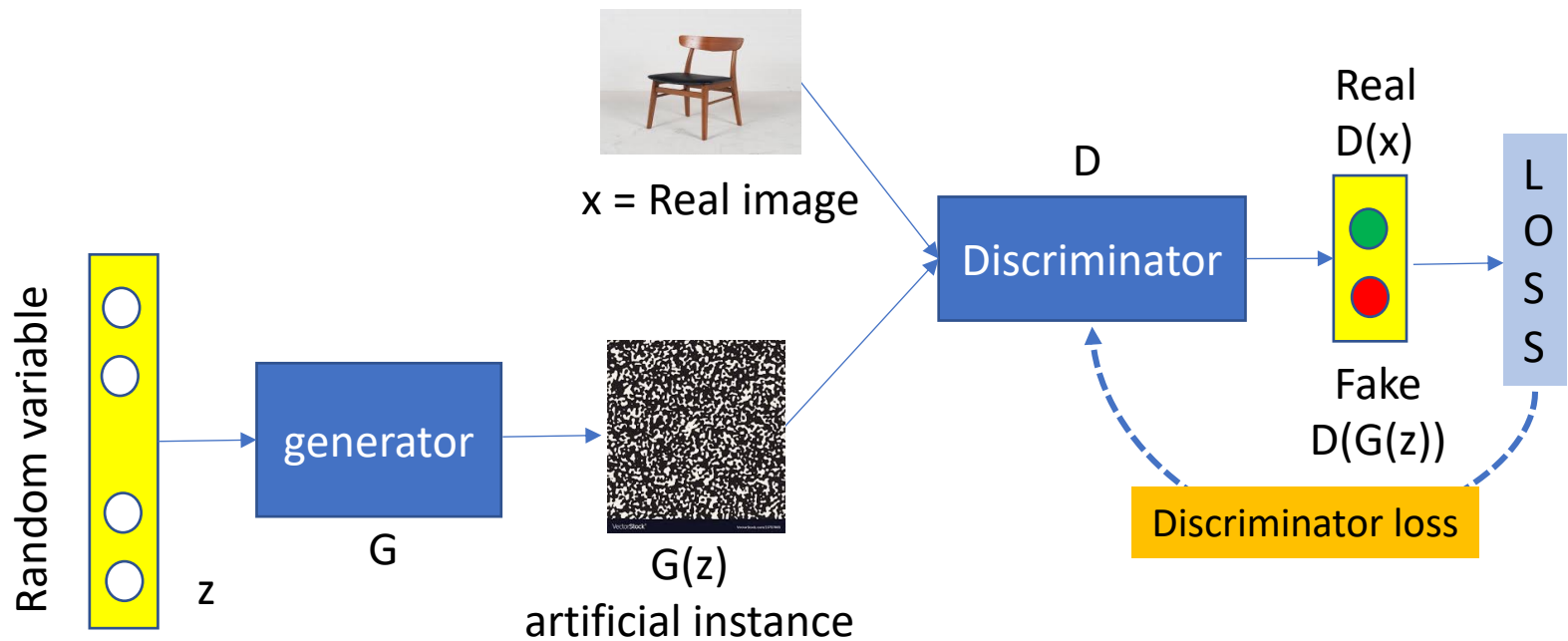


- Generator: generates fake samples to fool the Discriminator
- Discriminator: distinguishes between real and fake samples
- Train them against each other until we get better Generator and Discriminator



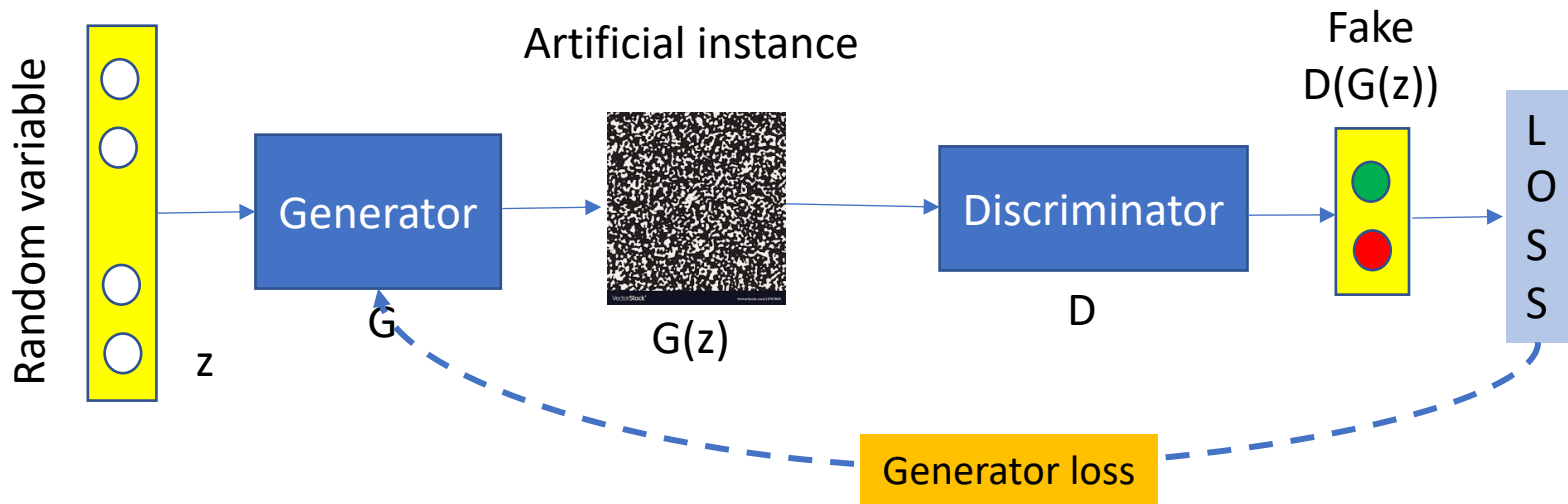
- x is real, if $D(x) = 0$, weights of Discriminator are modified
- $G(z)$ is fake, if $D(G(z)) = 1$, weights of Discriminator are modified
if $D(G(z)) = 0$, weights of Generator are modified
(usually $D(G(z)) \in (0,1)$, then both Discriminator and Generator need modified)
- Train them against each other until Discriminator (optimal) cannot distinguish real and fake samples (outputs 0.5)
- That is, fake samples look like real samples, F is similar to R .

Training the Discriminator



- Train with some real samples and artificial instances
- Modify the weights by backpropagation
- Minimize the classification errors

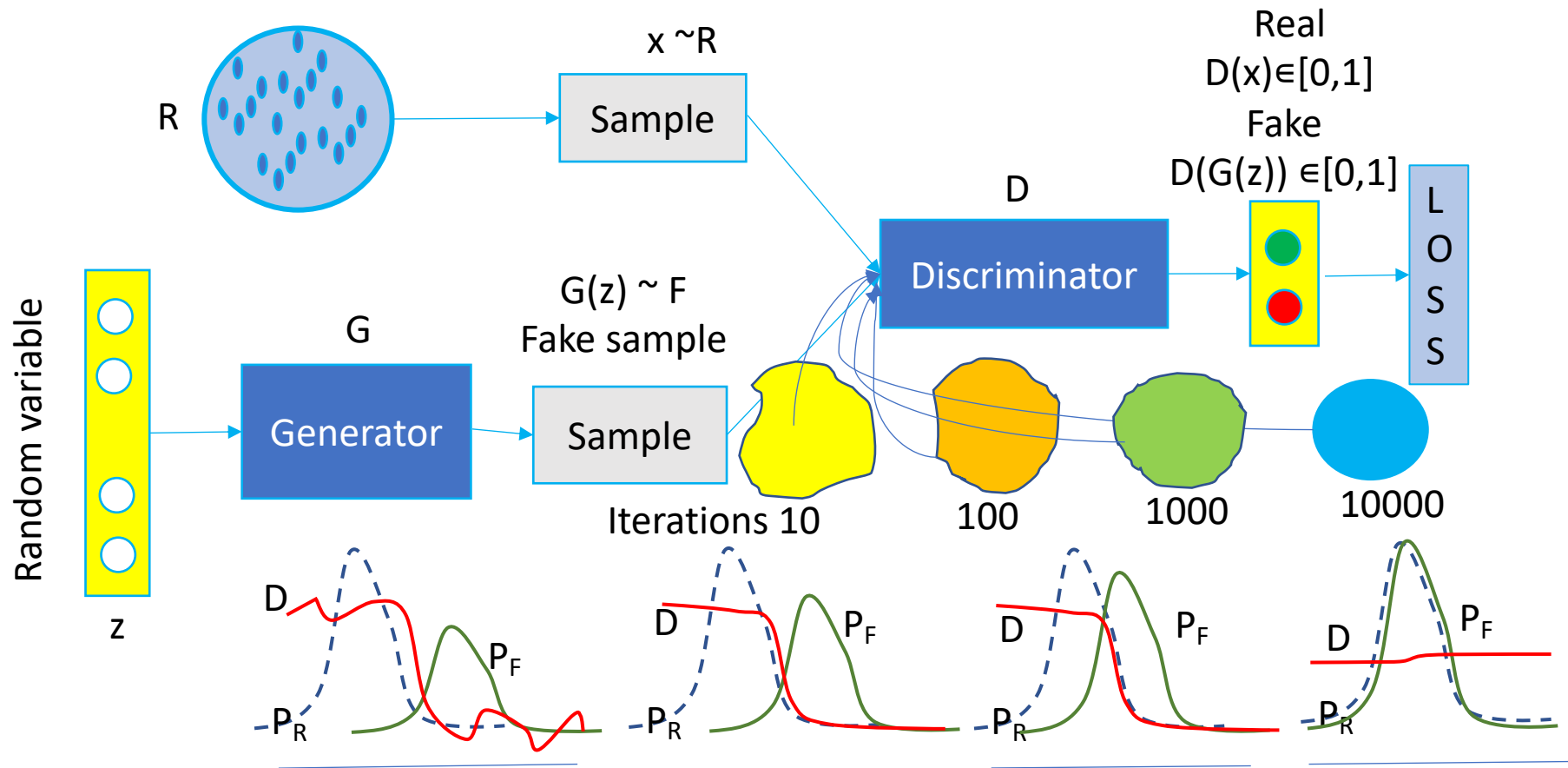
Training the Generator



- Fix the weights of Discriminator
- Generate an artificial instance, $G(z)$, and want to fool the Discriminator by expecting $D(G(z))=1$
- Weights of the generator will be modified by BP so that $G(z)$ is close to distribution R
- Maximize the Discriminator's errors

Generative Adversarial Network

- Generator: to generate data so that it can fool the Discriminator
- Discriminator: to distinguish between real data and fake data



When the generator can produce samples that fool the Discriminator, the output of Discriminator will be 0.5, i.e., can't tell real of fake.

Loss Function

For the Discriminator

→0 for real data →1

→0 for generated data →0

$$\max_D \{ \log (D(x)) + \log (1-D(G(z))) \}$$

Note: $D(.) \in [0,1]$
 $\log(D(.)) \in (-\infty, 0]$

For the Generator, we want to

$-\infty$ for generated data →1

$$\min_G \{ \log (D(x)) + \log (1-D(G(z))) \}$$

Combining both the Discriminator and Generator

$$\min_G \max_D \{ \log (D(x)) + \log (1-D(G(z))) \}$$

Training of GAN

Objective: $\min_G \max_D \{ E_{x \sim R} [\log (D(x))] + E_{z \sim Z} [\log (1 - D(G(z)))] \}$

Training by stochastic gradient descent

for k steps **do**

Training Discriminator

- Get m generated samples from noise samples $\{z_1, z_2, \dots, z_m\}$
- Get m real samples $\{x_1, x_2, \dots, x_m\}$ from real data
- Train the Discriminator by **ascending** the gradient (**maximize**)

$$\nabla_{\theta_D} \frac{1}{m} \sum_{i=1}^m [\log D(x_i) + \log(1 - D(G(z_i)))]$$

Training Generator

- Get m generated samples from noise samples $\{z_1, z_2, \dots, z_m\}$
- Train the Generator by **descending** the gradient (**minimize**)

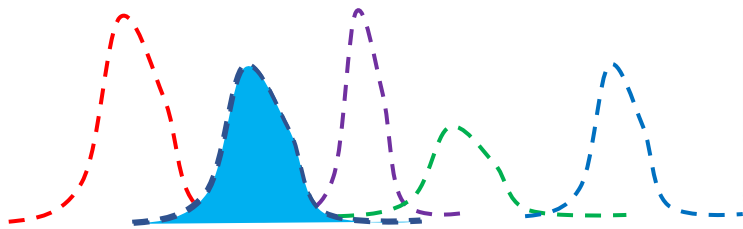
$$\nabla_{\theta_G} \frac{1}{m} \sum_{i=1}^m [\log D(x_i) + \log(1 - D(G(z_i)))]$$

Problems in GAN

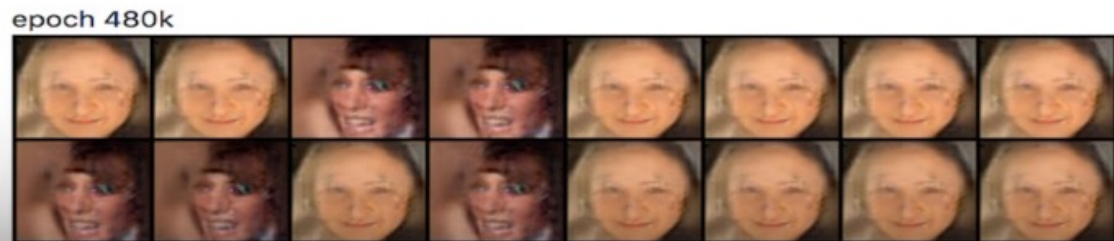
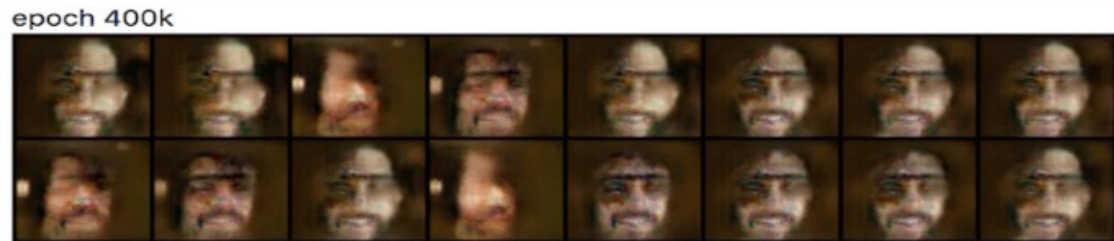
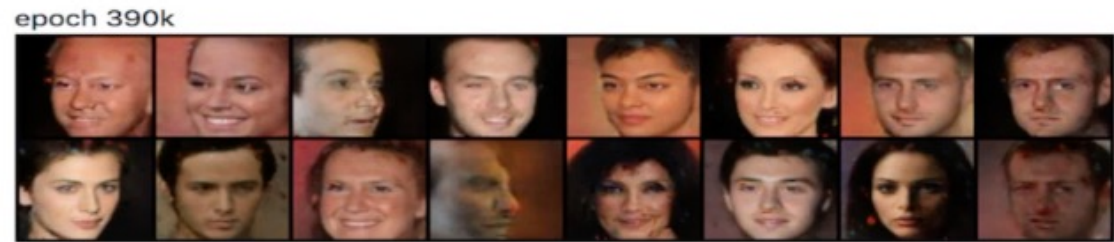
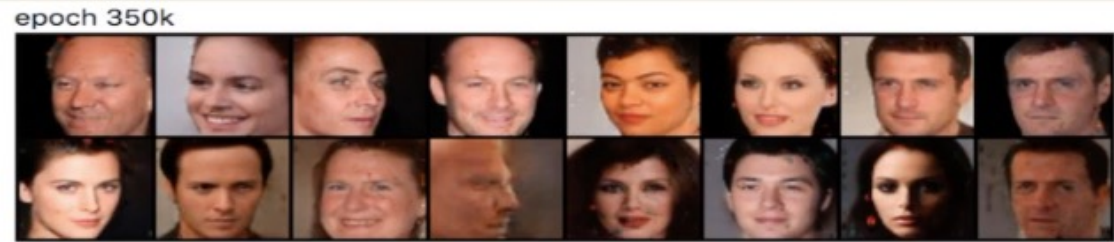
1) Mode Collapse

– the generator tends to produce one type of output

Real data contains multi-mode and generator tries to learn well on only one of them.



It is easier to be expert on one instead of “jack of all trades”

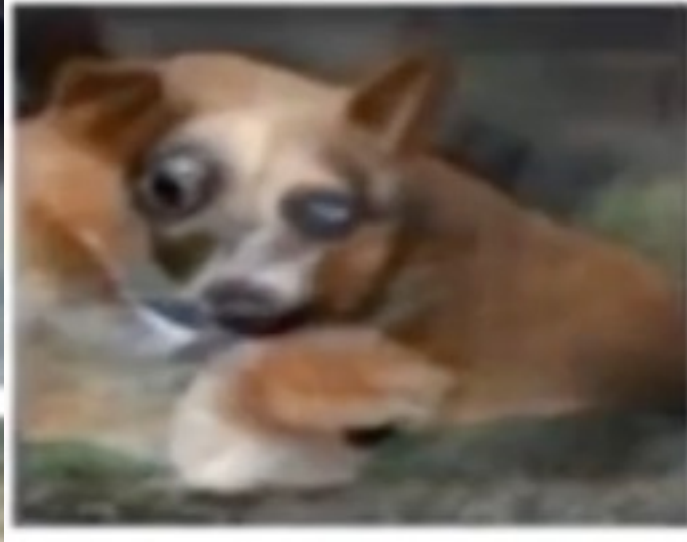


2) Non-Convergence

- GAN is based on the **zero-sum non-cooperative** game (**minimax game**)

Problems in GAN

3. distorted figures



Deep Generative Models

- Deep Dream
- Style transfer
- Autoencoder
- Variation Autoencoder (VAE)
- Generative Adversarial Network (GAN)
 - **Conditional-GAN**
 - Pix2Pix
 - CycleGAN
- Stable Diffusion

Conditional-GAN

M. Mirza, S. Osindero, [Conditional Generative Adversarial Nets](#) (2014).

- Weakness of GAN:
 - not able to generate data with the target label
 - the dataset generated lacks diversity.
- How can we control the type of images generated?
- Example:

written digits

0	1	2	3	4
5	6	7	8	9

Clothes:

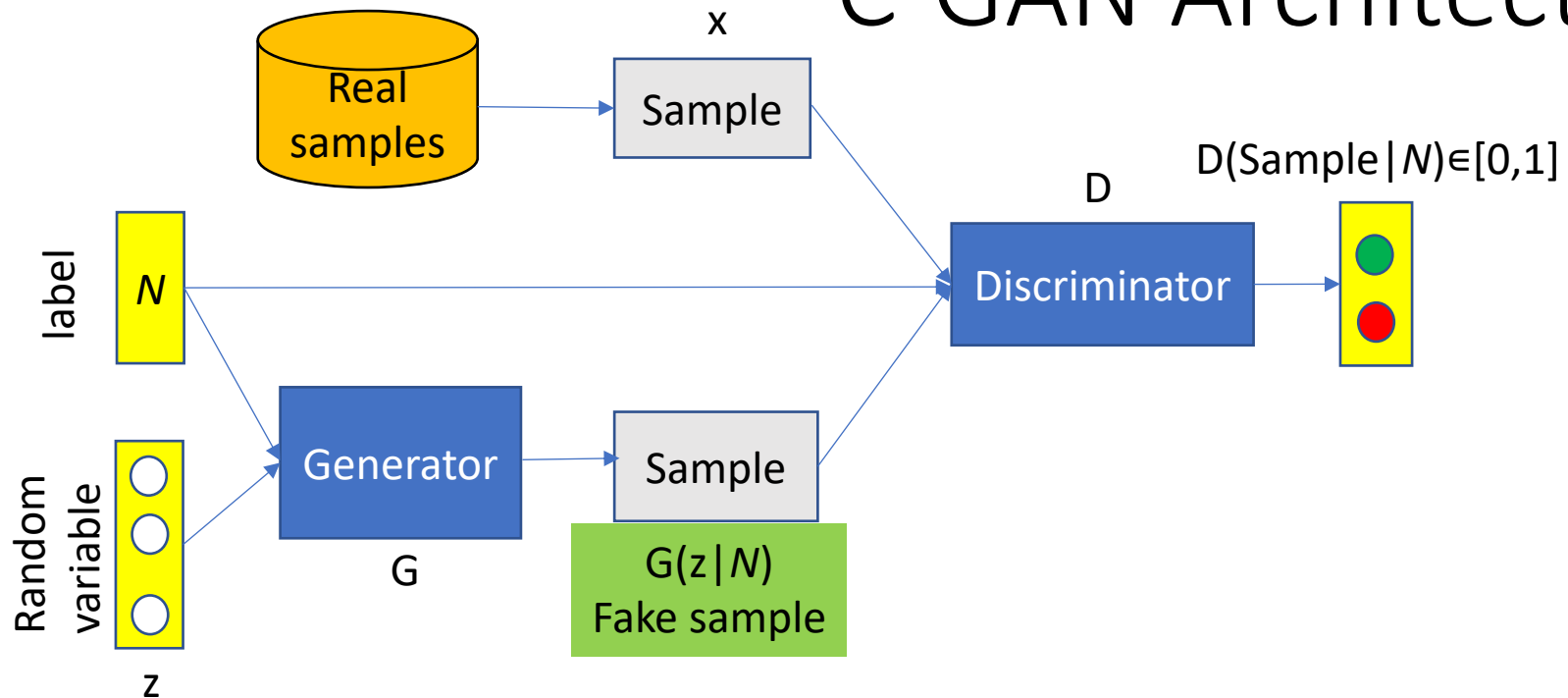
Dress
Suit
Blouse
T-shirt
Jacket
Jean
Sweater
Shirt
Coat
...



Conditional GAN

- Add a label N
 - to generator input to generate data
 - to discriminator input to distinguish real data better.
- Trained so that generator and discriminator are conditioned on N
- A trained GAN can generate images of a given label (type)
- Not totally unsupervised (need to label)

C-GAN Architecture



- There is a “complex” relationship between the random variable z (latent vector) and generated image $G(z)$
- **Generator:** Label N (1-hot vector) encodes (using embedding layer to concatenating with the seed) a latent vector for generating image.
- **Discriminator:** Label N using embedding layer (same size as image) to concatenation with input image.

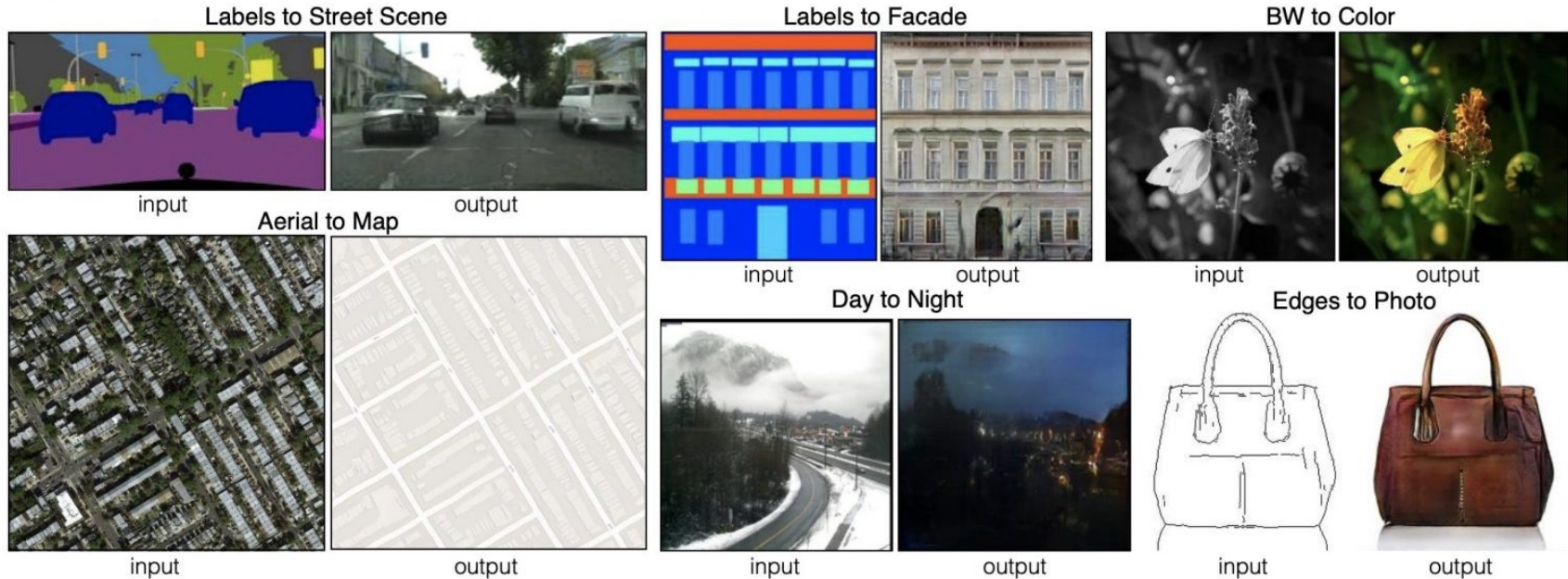
Deep Generative Models

- Deep Dream
- Style transfer
- Autoencoder
- Variation Autoencoder (VAE)
- Generative Adversarial Network (GAN)
 - Conditional-GAN
 - **Pix2Pix**
 - CycleGAN
- Stable Diffusion

Pix2Pix: Image-to-Image Translation

-- Application of C-GAN

P. Isola, et al, Image-to-Image Translation with Conditional Adversarial Networks, CVPR 2017



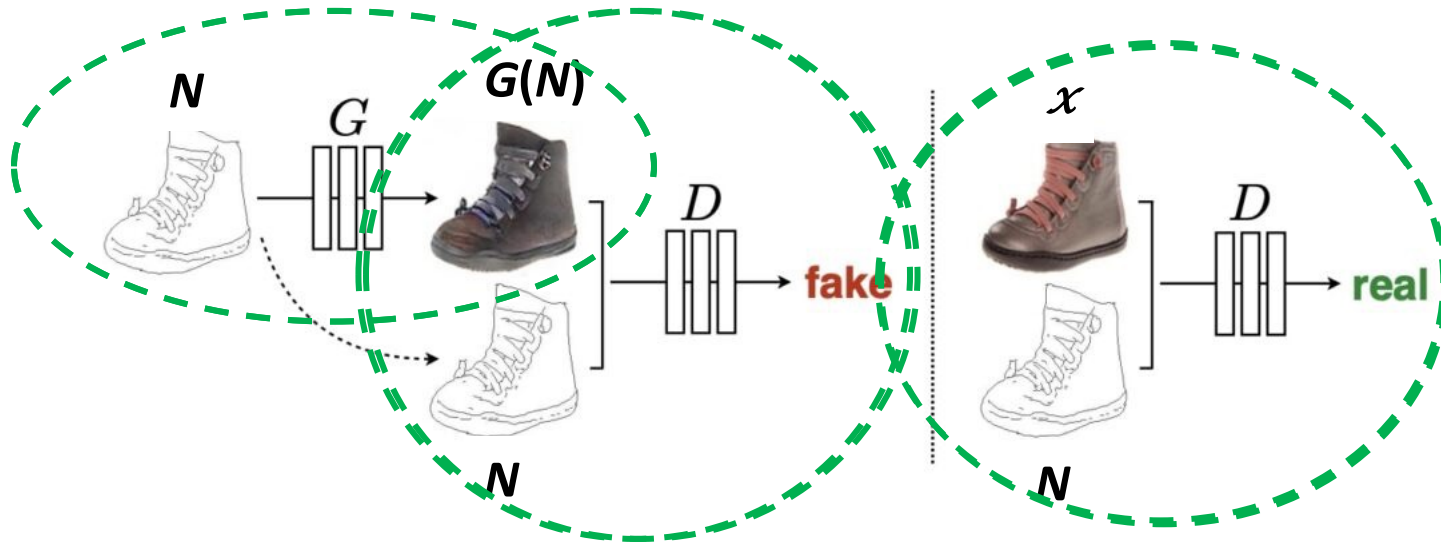
Pix2Pix Examples:

- segmentation map to RGB image (urban scene or facade)
- aerial or satellite view to a map
- Day to nighttime scenario
- BW image to colored image
- sketch to image

Applications of C-GAN, conditioned on input image content

Pix2Pix GAN

Extend C-GAN to perform Paired Image-to-Image translation



- Generator takes image N and outputs translated version $G(N)$.
- Discriminator is fed with a real image x or a fake (generated) image $G(N)$ conditioned on the same input image N
- Generator **learns** to produce image $G(N)$ similarly to image x based on UNet architecture as they have the same size.
- Discriminator **learns** to classify (1) image $G(N)$ as fake, and (2) real image x as real, both conditioned on input N

Deep Generative Models

- Deep Dream
- Style transfer
- Autoencoder
- Variation Autoencoder (VAE)
- Generative Adversarial Network (GAN)
 - Conditional-GAN
 - Pix2Pix
 - **CycleGAN**
- Stable Diffusion

CycleGAN

Zhu et al., Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks, ICCV 2017

Different from Pix2Pix

Without paired examples for training,
i.e., pictures before and after translation

(Unpaired Training Input)



Horse



Zebra



Photo

Painting

CycleGAN Architecture

Monet $\leftrightarrow$ Photos



Monet $\rightarrow$ photo

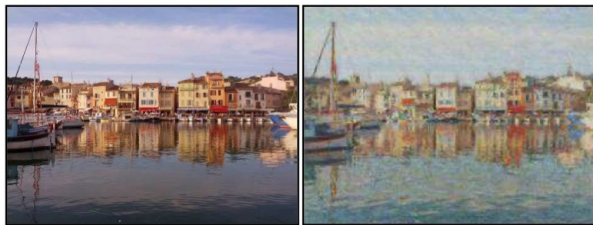


photo $\rightarrow$ Monet

Zebras $\leftrightarrow$ Horses

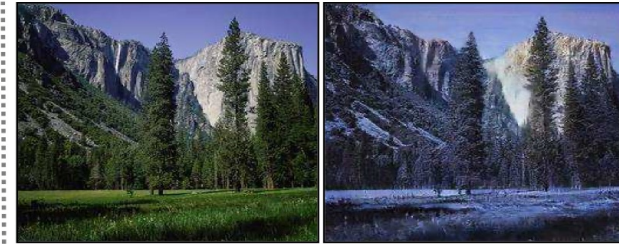


zebra $\rightarrow$ horse



horse $\rightarrow$ zebra

Summer $\leftrightarrow$ Winter



summer $\rightarrow$ winter



winter $\rightarrow$ summer

2 generators

Domain-Y $\rightarrow$ **Generator-X (G_x)** $\rightarrow$ Domain-X

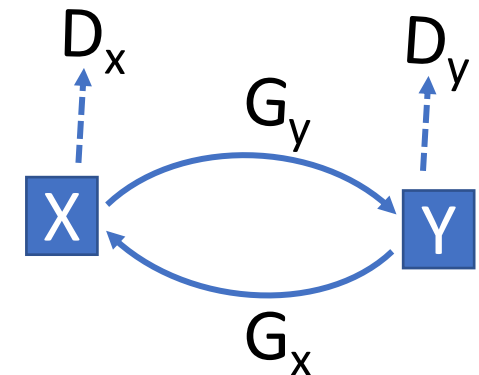
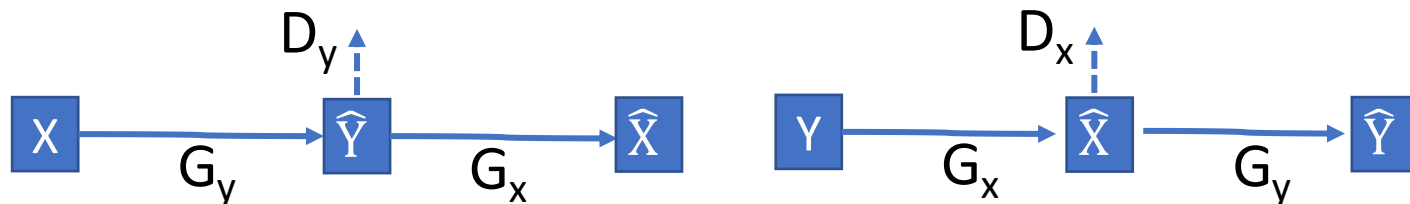
Domain-X $\rightarrow$ **Generator-Y (G_y)** $\rightarrow$ Domain-Y

2 discriminators

Domain-X $\rightarrow$ **Discriminator-X (D_x)** $\rightarrow$ [Real/Fake]

Domain-Y $\rightarrow$ **Discriminator-Y (D_y)** $\rightarrow$ [Real/Fake]

Cycle generation

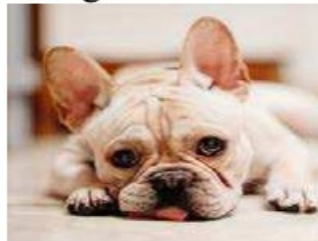


Limitations

- Good for color and texture changes (L1 norm)
- Little success for geometric changes
e.g., apple to orange, dog to cat
- Fails due to training data
no data with persons riding on horses or zebras.



apple → orange



dog → cat



horse → zebra